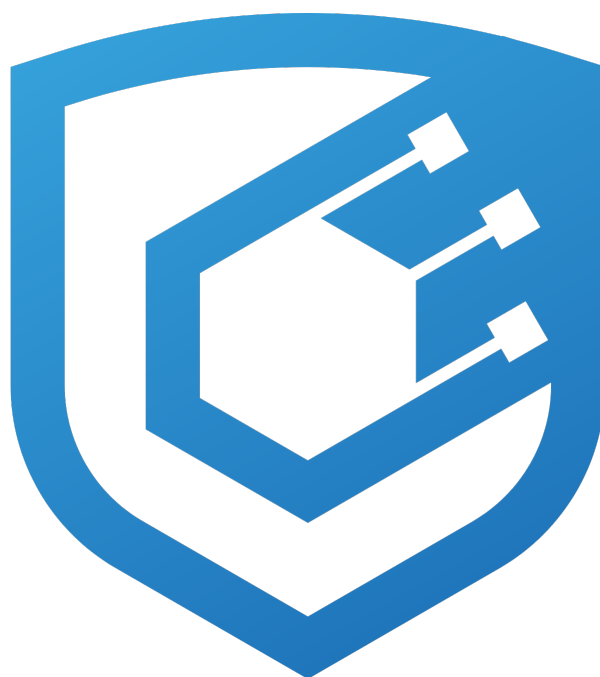




Spiral Stake V2 Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Carrotsmuggler](#)

[MrPotatoMagic](#)

March 12, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	3
7	Findings	7
7.1	Critical Risk	7
7.1.1	Arbitrary external call in <code>FlashLeverageRouter::swapAndLeverage</code> allows draining tokens via dangling approvals	7
7.2	High Risk	8
7.2.1	Arithmetic underflow in <code>FlashLeverage::withdrawCollateral</code> locks collateral after debt repayment on appreciated positions	8
7.2.2	Yield fee is charged on the full yield every withdrawal because <code>amountDepositedInLoanToken</code> is not updated after fee collection	12
7.3	Medium Risk	14
7.3.1	<code>FlashLeverage::withdrawCollateral</code> reverts with division-by-zero after full debt repayment via <code>FlashLeverage::repay</code>	14
7.3.2	Oracle price fluctuations allow users to bypass or be overcharged yield fees in <code>FlashLeverage::withdrawCollateral</code>	17
7.3.3	Deposit fee on non-correlated markets can be bypassed by leveraging through the swap router	18
7.3.4	<code>amountDepositedInLoanToken</code> can be inflated via swap router manipulation to bypass yield fees	19
7.3.5	Yield fee bypass during deleverage by routing collateral swap through an attacker-controlled pool	20
7.3.6	Malicious user can bypass yield fee by specifying non-zero <code>borrowShares</code> and inflated <code>amountRepay</code>	20
7.3.7	Incorrect loan to collateral token decimal scaling leads to zero yield fees	22
7.3.8	Native tokens not supported in the router	22
7.3.9	Owner can configure user proxies as swap routers to change Morpho positions and move user funds	23
7.4	Low Risk	24
7.4.1	Arithmetic underflow in <code>FlashLeverage::borrow</code> blocks borrowing capacity on correlated pairs after collateral appreciation	24
7.4.2	Anyone can call <code>FlashLeverage::leverage</code> on behalf of any account, enabling dust position griefing and block reorg attacks	27
7.4.3	<code>FlashLeverage::leverage</code> lacks reentrancy protection, allowing state manipulation during external swap calls	28
7.4.4	<code>withdrawCollateral</code> reverts on division by zero when <code>amountWithdrawInLoanToken</code> rounds to zero	29
7.4.5	No functionality to remove or deprecate supported markets	29
7.4.6	<code>amountDepositedInLoanToken</code> becomes stale after manual mode operations or Morpho liquidations	30
7.4.7	Unspent <code>tokenIn</code> not refunded after incomplete swaps in <code>SwapManager::_swapToken</code>	30
7.5	Informational	32
7.5.1	<code>FlashLeverage::borrow</code> missing correlated-pair guard allows calls on correlated markets	32
7.5.2	<code>LeveragePosition::amountCollateral</code> is not updated	35
7.5.3	Collateral/repayments supplied directly to Morpho bypasses <code>amountDepositedInLoanToken</code> tracking and is treated as yield	35

7.5.4	Deposit fee charged on <code>FlashLeverage::repay</code> but not on <code>FlashLeverage::deleverage</code> , creating a fee inconsistency	36
7.5.5	Yield fee <code>feeInCollateral</code> can round down to zero, allowing fee bypass via small repeated withdrawals	36
7.5.6	Pre-borrow LTV check underestimates actual post-borrow LTV due to Morpho's share-based rounding	37
7.5.7	Flash loan repayment in <code>_handleDeleverage</code> can consume stuck/dust loan tokens held by <code>FlashLeverage</code>	37
7.5.8	Incorrect comment in <code>_revertIfEffectiveLtvTooHigh</code> references <code>_handleDeleverage</code> instead of <code>_handleLeverage</code>	38
7.5.9	<code>FlashLeverage::getMaxLtv</code> underflows for markets with <code>lltv</code> below <code>LIQUIDATION_BUFFER</code>	38
7.5.10	<code>UserProxy</code> implementation contract can be initialized by anyone	38
7.5.11	Unused error <code>FlashLeverage__CannotBorrowForCorrelatedPair</code> in <code>FLibrary</code>	39
7.5.12	<code>FlashLeverageRouter::swapAndLeverage</code> uses full balance instead of swap output, capturing pre-existing tokens	39
7.5.13	Implement consistent variable naming across contracts	39
7.5.14	Missing event emission on critical state changes and operations	40
7.5.15	Consider using named mappings to improve code readability	41
7.5.16	<code>FlashLeverage</code> lacks pause functionality	42
7.6	Gas Optimization	43
7.6.1	Consider using custom errors instead of string error messages	43

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Spiral Stake is a non-custodial leveraged looping protocol that amplifies yields on yield-bearing assets using flash-loan leverage on Morpho lending markets.

The protocol works with any yield-bearing collateral that has a Morpho market - staked stablecoins like sUSDe, yield-bearing ETH like wstETH, and non-correlated directional markets like ETH/USD and BTC/USD.

The core idea is to exploit the spread between what user collateral earns and what it costs to borrow against it. When collateral yield is higher than the borrow rate, leverage amplifies that positive spread. Spiral Stake automates the entire loop into one atomic flash-loan transaction, so there's no multi-step execution risk and gas costs stay low.

5 Audit Scope

The audit scope was limited to:

```
src/core/FlashLeverage/FlashLeverage.sol
src/core/FlashLeverage/MarketPositionManager.sol
src/core/FlashLeverage/SwapManager.sol
src/core/FlashLeverage/UserProxy.sol
src/core/libraries/Math.sol
src/core/libraries/TokenHelper.sol
src/router/FlashLeverageRouter.sol
```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the [Spiral Stake V2](#) smart contracts provided by [Spiral Stake](#).

The findings consist of 1 Critical, 2 High, 9 Medium, and 7 Low severity issues, with the remainder being informational and gas optimizations.

Centralization Risks

The contract owner can perform the following functions:

- Pause `leverage`, `deleverage`, `increaseLeverage`, `supplyCollateral`, `borrow`, `repay`, `withdrawCollateral` operations - this was added as a recommendation from the auditors
- Add arbitrary Morpho markets (`addSupportedMarkets`)
- Whitelist or delist swap routers (`setSwapRouter`)
- Enable manual mode on any user's proxy (`enableManualMode`)
- Change treasury address (`updateTreasury`)
- Update yield fee from 0% to 10% (`updateYieldFee`)
- Update deposit fee from 0% to 1% (`updateDepositFee`)
- Ownership renunciation is intentionally disabled

Post Audit Recommendations

Even though the codebase was relatively small, hence the audit was short (only 4 working days) we still found 3 significant issues (1 Critical, 2 High). Hence it is recommended that prior to deploying significant capital on-chain in a production environment, another audit be conducted.

Summary

Project Name	Spiral Stake V2
Repository	v2-core
Commit	ead97aa2313a...
Fix Commit	11e15e7bad99...
Audit Timeline	Mar 2nd - Mar 5th, 2026
Methods	Manual Review

Issues Found

Critical Risk	1
High Risk	2
Medium Risk	9
Low Risk	7
Informational	16
Gas Optimizations	1
Total Issues	36

Summary of Findings

[C-1] Arbitrary external call in <code>FlashLeverageRouter::swapAndLeverage</code> allows draining tokens via dangling approvals	Resolved
[H-1] Arithmetic underflow in <code>FlashLeverage::withdrawCollateral</code> locks collateral after debt repayment on appreciated positions	Resolved
[H-2] Yield fee is charged on the full yield every withdrawal because <code>amountDepositedInLoanToken</code> is not updated after fee collection	Resolved
[M-1] <code>FlashLeverage::withdrawCollateral</code> reverts with division-by-zero after full debt repayment via <code>FlashLeverage::repay</code>	Resolved
[M-2] Oracle price fluctuations allow users to bypass or be overcharged yield fees in <code>FlashLeverage::withdrawCollateral</code>	Acknowledged
[M-3] Deposit fee on non-correlated markets can be bypassed by leveraging through the swap router	Acknowledged
[M-4] <code>amountDepositedInLoanToken</code> can be inflated via swap router manipulation to bypass yield fees	Acknowledged
[M-5] Yield fee bypass during deleverage by routing collateral swap through an attacker-controlled pool	Acknowledged
[M-6] Malicious user can bypass yield fee by specifying non-zero borrow-Shares and inflated <code>amountRepay</code>	Resolved
[M-7] Incorrect loan to collateral token decimal scaling leads to zero yield fees	Resolved
[M-8] Native tokens not supported in the router	Resolved
[M-9] Owner can configure user proxies as swap routers to change Morpho positions and move user funds	Resolved
[L-1] Arithmetic underflow in <code>FlashLeverage::borrow</code> blocks borrowing capacity on correlated pairs after collateral appreciation	Resolved
[L-2] Anyone can call <code>FlashLeverage::leverage</code> on behalf of any account, enabling dust position grieving and block reorg attacks	Resolved
[L-3] <code>FlashLeverage::leverage</code> lacks reentrancy protection, allowing state manipulation during external swap calls	Resolved
[L-4] <code>withdrawCollateral</code> reverts on division by zero when <code>amountWithdrawInLoanToken</code> rounds to zero	Resolved
[L-5] No functionality to remove or deprecate supported markets	Resolved
[L-6] <code>amountDepositedInLoanToken</code> becomes stale after manual mode operations or Morpho liquidations	Acknowledged
[L-7] Unspent <code>tokenIn</code> not refunded after incomplete swaps in <code>SwapManager::_swapToken</code>	Resolved
[I-01] <code>FlashLeverage::borrow</code> missing correlated-pair guard allows calls on correlated markets	Resolved
[I-02] <code>LeveragePosition::amountCollateral</code> is not updated	Resolved
[I-03] Collateral/repayments supplied directly to Morpho bypasses <code>amountDepositedInLoanToken</code> tracking and is treated as yield	Acknowledged

[I-04] Deposit fee charged on <code>FlashLeverage::repay</code> but not on <code>FlashLeverage::deleverage</code> , creating a fee inconsistency	Acknowledged
[I-05] Yield fee <code>feeInCollateral</code> can round down to zero, allowing fee bypass via small repeated withdrawals	Acknowledged
[I-06] Pre-borrow LTV check underestimates actual post-borrow LTV due to Morpho's share-based rounding	Acknowledged
[I-07] Flash loan repayment in <code>_handleDeleverage</code> can consume stuck/dust loan tokens held by <code>FlashLeverage</code>	Acknowledged
[I-08] Incorrect comment in <code>_revertIfEffectiveLtvTooHigh</code> references <code>_handleDeleverage</code> instead of <code>_handleLeverage</code>	Resolved
[I-09] <code>FlashLeverage::getMaxLtv</code> underflows for markets with <code>lltv</code> below <code>LIQUIDATION_BUFFER</code>	Acknowledged
[I-10] <code>UserProxy</code> implementation contract can be initialized by anyone	Resolved
[I-11] Unused error <code>FlashLeverage__CannotBorrowForCorrelatedPair</code> in <code>FLError</code> library	Resolved
[I-12] <code>FlashLeverageRouter::swapAndLeverage</code> uses full balance instead of swap output, capturing pre-existing tokens	Acknowledged
[I-13] Implement consistent variable naming across contracts	Resolved
[I-14] Missing event emission on critical state changes and operations	Resolved
[I-15] Consider using named mappings to improve code readability	Resolved
[I-16] <code>FlashLeverage</code> lacks pause functionality	Resolved
[G-1] Consider using custom errors instead of string error messages	Resolved

7 Findings

7.1 Critical Risk

7.1.1 Arbitrary external call in `FlashLeverageRouter::swapAndLeverage` allows draining tokens via dangling approvals

Description: In `FlashLeverageRouter::swapAndLeverage`, the caller supplies both `swapData.extRouter` and `swapData.extCalldata`, which are used in a raw `.call()` without any validation that the router is whitelisted:

```
_forceApprove(tokenIn, address(swapData.extRouter), amountIn);  
(bool success, ) = swapData.extRouter.call(swapData.extCalldata);  
require(success, "Swap Router Call Failed");
```

Unlike `SwapManager::_swapToken` in the core `FlashLeverage` contract, which checks `s_isSwapRouter[swapData.extRouter]`, the router contract performs no such check. This means a malicious user can:

1. Point `swapData.extRouter` to any arbitrary contract, including a token contract.
2. Craft `swapData.extCalldata` to call `transferFrom` on that token, draining any tokens that other users have previously approved to the `FlashLeverageRouter`.

Since `_forceApprove` grants the attacker-controlled address an approval on `tokenIn`, and the raw call can target any contract with any calldata, an attacker has full control to execute arbitrary logic in the context of the router's approvals and balances.

Impact: Critical loss of funds. Any user who has granted token approvals to the `FlashLeverageRouter` contract (e.g., for a previous or future `swapAndLeverage` call) can have their tokens stolen. An attacker crafts a malicious `extRouter` and `extCalldata` to call `transferFrom` on the approved token, transferring the victim's tokens to themselves.

Proof of Concept:

1. Alice calls `swapAndLeverage` normally. Her `tokenIn` approval to `FlashLeverageRouter` may still be active (common pattern: `approve(type(uint256).max)`).
2. Bob calls `swapAndLeverage` with:
 - `tokenIn` = any token (e.g., a worthless token or `amountIn = 1 wei`)
 - `swapData.extRouter` = address of Alice's approved token
 - `swapData.extCalldata` = `abi.encodeCall(IERC20.transferFrom, (alice, bob, aliceBalance))`
3. The raw `.call()` executes `transferFrom` on the token contract, transferring Alice's tokens to Bob.
4. The call succeeds, and Bob drains Alice's funds.

Recommended Mitigation: Validate the external router against a whitelist, similar to how `SwapManager` does it in the core contract. Either maintain an independent whitelist in `FlashLeverageRouter` or query the `FlashLeverage` contract's `isValidSwapRouter`:

SpiralStake: Fixed in [a08729](#).

Cyfrin: Verified. Fixed by verifying router is whitelisted.

7.2 High Risk

7.2.1 Arithmetic underflow in `FlashLeverage::withdrawCollateral` locks collateral after debt repayment on appreciated positions

Description: On correlated markets, when a user repays debt via `FlashLeverage::repay` and then attempts to withdraw collateral via `FlashLeverage::withdrawCollateral`, the transaction reverts with an arithmetic underflow:

```
position.amountDepositedInLoanToken -= amountWithdrawInLoanToken;
```

`amountDepositedInLoanToken` accumulates capital contributions at **historical** oracle prices:

- Initial collateral value during `FlashLeverage::leverage` (line 670)
- Repaid debt amount during `FlashLeverage::repay` (line 417)
- After full repayment: `amountDepositedInLoanToken = initialDeposit + debtValue * originalPrice`

But `amountWithdrawInLoanToken` is computed at the *current* oracle price (line 438):

```
uint256 amountWithdrawInLoanToken = getCollateralValueInLoanToken(market, amountWithdraw);
```

With any collateral appreciation, the current-price value of the full collateral exceeds the historical-price tracker, and the subtraction underflows.

Concrete walkthrough (weETH/WETH market, 5% appreciation):

Setup

- User deposits 10 weETH as collateral (worth 10 WETH)
- Opens a 3x leveraged position at ~70% LTV
- Position: 30 weETH collateral in Morpho, 20 WETH debt
- `amountDepositedInLoanToken = 10 WETH`

Time passes, weETH appreciates 5% (1 weETH = 1.05 WETH)

User repays debt

User calls `FlashLeverage::repay(USER, 0, 20 WETH, borrowShares):`

- Debt: 20 WETH → 0
- `amountDepositedInLoanToken += 20` → now 30 WETH

This is correct — the user has contributed 30 WETH of capital total (10 initial + 20 repaid).

User tries to withdraw all collateral

```
amountDepositedInLoanToken = 30 WETH           (accumulated at historical prices)
amountWithdrawInLoanToken  = 30 × 1.05 = 31.5 WETH (valued at current price)

position.amountDepositedInLoanToken -= amountWithdrawInLoanToken
// 30 - 31.5 → underflow, reverts
```

The user can only withdraw up to $30 / 1.05 = 28.57$ weETH. The remaining ~1.43 weETH (worth 1.5 WETH — exactly the yield) is permanently locked in Morpho with zero debt and no withdrawal path.

Impact

Loss of user funds. After repaying debt on a correlated-market position with any collateral appreciation, a portion of the user's collateral becomes irretrievable. The locked amount equals the yield and scales with the degree of appreciation. There is no debt to liquidate and no alternative withdrawal mechanism, so the funds are permanently stuck in Morpho.

Proof of Concept

Place the following test in test/ (e.g. test/RepayInflatesYieldBaseline.t.sol):

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC20Metadata} from "@openzeppelin/contracts/interfaces/IERC20Metadata.sol";
import {IMorpho, MarketParams, Id, Position} from "@morpho/interfaces/IMorpho.sol";
import {IOracle} from "@morpho/interfaces/IOracle.sol";

import {FlashLeverage} from "src/core/FlashLeverage/FlashLeverage.sol";
import {MarketConfig} from "src/core/structs/MarketConfig.sol";
import {LeverageParams} from "src/core/structs/LeverageParams.sol";
import {SwapData} from "src/core/structs/SwapData.sol";
import {LeveragePosition} from "src/core/structs/LeveragePosition.sol";
import {Math} from "src/core/libraries/Math.sol";

import {MockExtRouter} from "../mocks/MockExtRouter.sol";
import {MockOracle} from "../mocks/MockOracle.sol";

/// @title PoC - Arithmetic underflow in withdrawCollateral locks collateral
/// @notice After repaying debt on an appreciated correlated-market position,
///         FlashLeverage::withdrawCollateral reverts with an arithmetic underflow at
///         FlashLeverage.sol:494:
///
///         position.amountDepositedInLoanToken -= amountWithdrawInLoanToken;
///
///         amountDepositedInLoanToken accumulates at historical oracle prices,
///         but amountWithdrawInLoanToken is valued at the current (higher) price.
///         With any appreciation, the withdrawal value exceeds the tracker,
///         causing an underflow. The collateral is permanently locked in Morpho
///         with zero debt and no withdrawal path.
contract RepayInflatesYieldBaseline is Test {
    using Math for uint256;

    // --- Mainnet constants ---
    address constant MORPHO = 0xBbBbBbBbBb9cC5e90e3b3Af64bdAF62C37EEFFCb;
    address constant TREASURY = 0xeB90258b1F74a846F7941514C7c02Bb03EB249D5;
    bytes32 constant MARKET_ID = 0x37e7484d642d90f14451f1910ba4b7b8e4c3ccdd0ec28f8b2bdb35479e472ba7;

    // --- State ---
    FlashLeverage fl;
    IMorpho morpho;
    MockExtRouter mockRouter;
    MarketParams market;
    address user;
    uint8 loanDecimals;

    uint256 constant COLLATERAL = 10_000e18;
    uint256 constant TARGET_LTV = 70e16;

    function setUp() public {
        // Fork mainnet
        vm.createSelectFork(vm.envString("ETH_RPC_URL"), vm.envUint("BLOCK_NUMBER"));

        morpho = IMorpho(MORPHO);
        market = morpho.idToMarketParams(Id.wrap(MARKET_ID));
        loanDecimals = IERC20Metadata(market.loanToken).decimals();

        // Deploy FlashLeverage
    }
}
```

```

fl = new FlashLeverage(MORPHO, TREASURY);

MarketConfig[] memory configs = new MarketConfig[](1);
configs[0] = MarketConfig({marketId: MARKET_ID, isCorrelated: true});
fl.addSupportedMarkets(configs);

// Deploy and whitelist mock swap router
mockRouter = new MockExtRouter();
fl.setSwapRouter(address(mockRouter), true);

// Seed Morpho with loan-token liquidity
address supplier = makeAddr("Supplier");
deal(market.loanToken, supplier, 100_000e18);
vm.startPrank(supplier);
IERC20(market.loanToken).approve(MORPHO, 100_000e18);
morpho.supply(market, 100_000e18, 0, supplier, "");
vm.stopPrank();

user = makeAddr("user");
}

function test_underflow_locksCollateralAfterRepay() external {
    // --- 1. Open leveraged position ---
    _openLeveragedPosition();

    // --- 2. Simulate 5% collateral appreciation ---
    _appreciateCollateral(105);

    // --- 3. Fully repay debt ---
    _repayAllDebt();

    // --- 4. Confirm zero debt, collateral remains ---
    LeveragePosition memory pos = fl.getUserLeveragePosition(user, 0);
    Position memory mPosAfter = morpho.position(Id.wrap(MARKET_ID), pos.userProxy);
    assertEq(mPosAfter.borrowShares, 0, "Debt should be zero");
    assertGt(mPosAfter.collateral, 0, "Collateral should remain");

    // --- 5. Attempt to withdraw all collateral - underflow reverts ---
    // Even collateral - 1 fails because the appreciated loan-token value
    // exceeds amountDepositedInLoanToken (set at the original price).
    vm.prank(user);
    vm.expectRevert(); // panic: arithmetic underflow (0x11) at line 494
    fl.withdrawCollateral(0, mPosAfter.collateral - 1);

    // --- 6. Show that a smaller withdrawal succeeds, proving partial lockup ---
    // Only ~95% (100/105) of collateral can be withdrawn; the rest is stuck.
    uint256 safeAmount = mPosAfter.collateral * 100 / 105;
    vm.prank(user);
    fl.withdrawCollateral(0, safeAmount);

    // Remaining collateral is locked in Morpho with zero debt
    Position memory mPosFinal = morpho.position(Id.wrap(MARKET_ID), pos.userProxy);
    assertGt(mPosFinal.collateral, 0, "Collateral permanently locked in Morpho");
    assertEq(mPosFinal.borrowShares, 0, "No debt - no liquidation path to recover funds");
}

// ---- Helpers ----

function _openLeveragedPosition() internal {
    deal(market.collateralToken, user, COLLATERAL);

    uint256 flashLoan = _calcFlashLoan(TARGET_LTV, COLLATERAL);
    SwapData memory swap = _buildSwap(market.loanToken, market.collateralToken, flashLoan);

```

```

vm.startPrank(user);
IERC20(market.collateralToken).approve(address(fl), COLLATERAL);
fl.leverage(
    user,
    LeverageParams({
        marketId: MARKET_ID,
        amountCollateral: COLLATERAL,
        amountFlashLoan: flashLoan,
        swapData: swap,
        minTokenOut: 0
    })
);
vm.stopPrank();
}

function _appreciateCollateral(uint256 pctOf100) internal {
    uint256 originalPrice = IOracle(market.oracle).price();
    uint256 appreciatedPrice = originalPrice * pctOf100 / 100;
    vm.mockCall(
        market.oracle,
        abi.encodeWithSelector(IOracle.price.selector),
        abi.encode(appreciatedPrice)
    );
}

function _repayAllDebt() internal {
    LeveragePosition memory pos = fl.getUserLeveragePosition(user, 0);
    Position memory mPos = morpho.position(Id.wrap(MARKET_ID), pos.userProxy);

    uint256 debt = fl.getSharesValueInLoanToken(market, mPos.borrowShares);
    deal(market.loanToken, user, debt);

    vm.startPrank(user);
    IERC20(market.loanToken).approve(address(fl), debt);
    fl.repay(user, 0, debt, mPos.borrowShares);
    vm.stopPrank();
}

function _calcFlashLoan(uint256 ltv, uint256 collateral) internal view returns (uint256) {
    uint256 colVal = fl.getCollateralValueInLoanToken(market, collateral)
        .scaleTo(loanDecimals, Math.STANDARD_DECIMALS);
    uint256 totalPos = colVal.divDown(Math.ONE - ltv);
    return (totalPos - colVal).scaleTo(Math.STANDARD_DECIMALS, loanDecimals);
}

function _buildSwap(address tokenIn, address tokenOut, uint256 amountIn) internal returns (SwapData
    memory) {
    uint256 oraclePrice = IOracle(market.oracle).price();
    uint256 amountOut = tokenIn == market.loanToken
        ? (amountIn * 1e36) / oraclePrice
        : (amountIn * oraclePrice) / 1e36;

    deal(tokenOut, address(mockRouter), amountOut);

    return SwapData({
        extRouter: address(mockRouter),
        extCalldata: abi.encodeCall(MockExtRouter.swap, (tokenIn, tokenOut, amountIn, amountOut))
    });
}
}

```

Run with: `forge test --match-test test_underflow_locksCollateralAfterRepay`

Recommended Mitigation

The root cause is the price mismatch: `amountDepositedInLoanToken` accumulates at historical prices but is decremented at the current price.

The `amountDeposited` must be stored in a price-independent manner. For example, say `wsteth-weth` market is being used. `wsteth` is 1.1 eth and grows to 1.2 eth.

During deposit of 1 `wsteth`, the `amountDeposited` must increment it by 1 `wsteth`. During withdrawals, it should decrement it by 1 `wsteth`, there can be no underflows in such a situation.

But when calculating the yield, `amountDepositedInLoanTokens` can be used, denominated in WETH (loan token). During deposit, `amountDepositedInLoanTokens` = 1.1 eth, during withdrawal, `amountDepositedInLoanTokens` = 1.2 eth, and the difference gives the yield. Basically, the amount-tracking and yield-tracking need to be decoupled.

SpiralStake: Fixed in [09b278](#).

Cyfrin: Verified.

7.2.2 Yield fee is charged on the full yield every withdrawal because `amountDepositedInLoanToken` is not updated after fee collection

Description: In `FlashLeverage::withdrawCollateral`, when a correlated position has generated yield (`netPositionValue > position.amountDepositedInLoanToken`), the contract charges a yield fee. When the withdrawal amount exceeds the yield generated, the fee is calculated on the entire `yieldGenerated` amount:

```
if (amountWithdrawInLoanToken > yieldGenerated) {
    yieldFeeInLoanToken = yieldGenerated.mulDown(s_yieldFee);
} else {
    yieldFeeInLoanToken = amountWithdrawInLoanToken.mulDown(s_yieldFee);
}
```

However, after this fee is charged, `position.amountDepositedInLoanToken` is only reduced by `amountWithdrawInLoanToken` (the withdrawal amount in loan token terms) at the end of the function. No adjustment is made to account for the yield that was already taxed. This means `yieldGenerated` (the difference between `netPositionValue` and `amountDepositedInLoanToken`) does not shrink by the fee amount that was already collected.

On subsequent withdrawals, the yield fee is recalculated from scratch against whatever yield exists at that moment — but yield that was previously taxed is still counted as untaxed yield, causing the protocol to double-charge (or more) yield fees across multiple partial withdrawals.

Impact: Users with correlated positions who perform multiple partial withdrawals are overcharged on yield fees. The same yield is taxed repeatedly on every withdrawal, resulting in a direct loss of funds for users. A user performing N partial withdrawals could pay up to N times the intended yield fee on overlapping yield portions.

Proof of Concept: Assume `s_yieldFee` = 10% for simplicity.

1. User opens a correlated leveraged position with `amountDepositedInLoanToken` = 100.
2. Over time, the position's `netPositionValue` grows to 120, so `yieldGenerated` = 20.
3. User calls `withdrawCollateral` to withdraw an amount worth 25 in loan tokens. Since `amountWithdrawInLoanToken` (25) > `yieldGenerated` (20), the IF branch is taken: fee is charged on the full yield → $\text{fee} = 20 * 10\% = 2$. After the function, `amountDepositedInLoanToken` is reduced by 25, becoming 75. The taxed yield is not accounted for.
4. Now `netPositionValue` 95, `amountDepositedInLoanToken` = 75, so `yieldGenerated` = 20 again — it hasn't changed because both values dropped by the same 25.
5. User calls `withdrawCollateral` again for another 25. The IF branch is taken again: $\text{fee} = 20 * 10\% = 2$. `amountDepositedInLoanToken` becomes 50.
6. Again `netPositionValue` 70, `amountDepositedInLoanToken` = 50, `yieldGenerated` = 20. A third withdrawal of 25 charges $\text{fee} = 20 * 10\% = 2$ once more.

After 3 withdrawals, the user has paid 6 in yield fees (3×2) on only 20 of actual yield — 3× the intended fee of 2. Every additional withdrawal re-taxes the same yield.

Recommended Mitigation: After charging the yield fee in the `amountWithdrawInLoanToken > yieldGenerated` branch, increase `amountDepositedInLoanToken` by the yield that was taxed (net of fees), so it is not counted again.

Spiral Stake: Fixed in [09b278](#).

Cyfrin: Verified.

7.3 Medium Risk

7.3.1 FlashLeverage::withdrawCollateral reverts with division-by-zero after full debt repayment via FlashLeverage::repay

Description: When a user fully repays their debt via FlashLeverage::repay and then attempts to withdraw all collateral via FlashLeverage::withdrawCollateral, the transaction reverts with a division-by-zero panic.

Inside FlashLeverage::withdrawCollateral, the collateral is first removed from Morpho (line 435), then FlashLeverage::_revertIfEffectiveLtvTooHigh is called (line 436) to validate the position's health. After the Morpho withdrawal, the proxy's position has zero collateral and zero borrow shares. The function reads these values and computes:

```
// FlashLeverage.sol:827
uint256 effectiveLtv = amountLoan.divDown(amountCollateralInLoanToken);
// = 0.divDown(0)
```

Math::divDown performs $(a * \text{ONE}) / b$ inside an unchecked block (lines:31-33). While the unchecked block disables overflow checks, division by zero is an EVM-level panic (error code 0x12) that cannot be suppressed. The transaction always reverts.

This means any user who repays their full debt cannot withdraw their remaining collateral in a single call. The collateral is stranded in Morpho.

Impact

After fully repaying debt, the user cannot withdraw all their collateral through FlashLeverage::withdrawCollateral. If the user calls withdraw for amount, where amount is their total balance, the function reverts. If they call withdraw for (amount-1), the function passes.

Proof of Concept

Place the following test in test/ (e.g. test/DivisionByZero.t.sol):

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC20Metadata} from "@openzeppelin/contracts/interfaces/IERC20Metadata.sol";
import {IMorpho, MarketParams, Id, Position} from "@morpho/interfaces/IMorpho.sol";
import {IOracle} from "@morpho/interfaces/IOracle.sol";

import {FlashLeverage} from "src/core/FlashLeverage/FlashLeverage.sol";
import {MarketConfig} from "src/core/structs/MarketConfig.sol";
import {LeverageParams} from "src/core/structs/LeverageParams.sol";
import {SwapData} from "src/core/structs/SwapData.sol";
import {LeveragePosition} from "src/core/structs/LeveragePosition.sol";
import {Math} from "src/core/libraries/Math.sol";

import {MockExtRouter} from "../mocks/MockExtRouter.sol";

/// @title PoC - Division by zero in _revertIfEffectiveLtvTooHigh
/// @notice After fully repaying debt, withdrawing all collateral triggers a
///         division by zero in FlashLeverage::_revertIfEffectiveLtvTooHigh (FlashLeverage.sol:827)
///         because both amountLoan and amountCollateralInLoanToken are zero.
///         The user's collateral is trapped in Morpho with no debt and no
///         withdrawal path.
contract DivisionByZero is Test {
    using Math for uint256;

    // --- Mainnet constants ---
    address constant MORPHO = 0xBbBbBbBbBb9cC5e90e3b3Af64bdAF62C37EEFFCb;
    address constant TREASURY = 0xeB90258b1F74a846F7941514C7c02Bb03EB249D5;
```

```

bytes32 constant MARKET_ID = 0x37e7484d642d90f14451f1910ba4b7b8e4c3ccdd0ec28f8b2bdb35479e472ba7;

// --- State ---
FlashLeverage fl;
IMorpho morpho;
MockExtRouter mockRouter;
MarketParams market;
address user;
uint8 loanDecimals;

uint256 constant COLLATERAL = 10_000e18;
uint256 constant TARGET_LTV = 70e16;

function setUp() public {
    // Fork mainnet
    vm.createSelectFork(vm.envString("ETH_RPC_URL"), vm.envUint("BLOCK_NUMBER"));

    morpho = IMorpho(MORPHO);
    market = morpho.idToMarketParams(Id.wrap(MARKET_ID));
    loanDecimals = IERC20Metadata(market.loanToken).decimals();

    // Deploy FlashLeverage
    fl = new FlashLeverage(MORPHO, TREASURY);

    MarketConfig[] memory configs = new MarketConfig[](1);
    configs[0] = MarketConfig({marketId: MARKET_ID, isCorrelated: true});
    fl.addSupportedMarkets(configs);

    // Deploy and whitelist mock swap router
    mockRouter = new MockExtRouter();
    fl.setSwapRouter(address(mockRouter), true);

    // Seed Morpho with loan-token liquidity
    address supplier = makeAddr("Supplier");
    deal(market.loanToken, supplier, 100_000e18);
    vm.startPrank(supplier);
    IERC20(market.loanToken).approve(MORPHO, 100_000e18);
    morpho.supply(market, 100_000e18, 0, supplier, "");
    vm.stopPrank();

    user = makeAddr("user");
}

function test_divisionByZero_withdrawAfterFullRepay() external {
    // --- Open leveraged position ---
    deal(market.collateralToken, user, COLLATERAL);

    uint256 flashLoan = _calcFlashLoan(TARGET_LTV, COLLATERAL);
    SwapData memory swap = _buildSwap(market.loanToken, market.collateralToken, flashLoan);

    vm.startPrank(user);
    IERC20(market.collateralToken).approve(address(fl), COLLATERAL);
    fl.leverage(
        user,
        LeverageParams({
            marketId: MARKET_ID,
            amountCollateral: COLLATERAL,
            amountFlashLoan: flashLoan,
            swapData: swap,
            minTokenOut: 0
        })
    );
    vm.stopPrank();
}

```



```

// --- Fully repay debt ---
LeveragePosition memory pos = fl.getUserLeveragePosition(user, 0);
Position memory mPos = morpho.position(Id.wrap(MARKET_ID), pos.userProxy);

uint256 debt = fl.getSharesValueInLoanToken(market, mPos.borrowShares);
deal(market.loanToken, user, debt);

vm.startPrank(user);
IERC20(market.loanToken).approve(address(fl), debt);
fl.repay(user, 0, debt, mPos.borrowShares);
vm.stopPrank();

// --- Confirm debt is zero, collateral remains ---
Position memory mPosAfter = morpho.position(Id.wrap(MARKET_ID), pos.userProxy);
assertEq(mPosAfter.borrowShares, 0, "Debt should be zero");
assertGt(mPosAfter.collateral, 0, "Collateral should remain");

// --- Attempt to withdraw all collateral - reverts with division by zero ---
vm.prank(user);
vm.expectRevert(); // panic: division by zero (0x12) in Math::divDown(0, 0)
fl.withdrawCollateral(0, mPosAfter.collateral);
}

// ---- Helpers (inlined, no external inheritance) ----

function _calcFlashLoan(uint256 ltv, uint256 collateral) internal view returns (uint256) {
    uint256 colVal = fl.getCollateralValueInLoanToken(market, collateral)
        .scaleTo(loanDecimals, Math.STANDARD_DECIMALS);
    uint256 totalPos = colVal.divDown(Math.ONE - ltv);
    return (totalPos - colVal).scaleTo(Math.STANDARD_DECIMALS, loanDecimals);
}

function _buildSwap(address tokenIn, address tokenOut, uint256 amountIn) internal returns (SwapData
↳ memory) {
    uint256 oraclePrice = IOracle(market.oracle).price();
    uint256 amountOut = tokenIn == market.loanToken
        ? (amountIn * 1e36) / oraclePrice
        : (amountIn * oraclePrice) / 1e36;

    deal(tokenOut, address(mockRouter), amountOut);

    return SwapData({
        extRouter: address(mockRouter),
        extCalldata: abi.encodeCall(MockExtRouter.swap, (tokenIn, tokenOut, amountIn, amountOut))
    });
}
}

```

Run with: `forge test --match-test test_divisionByZero_withdrawAfterFullRepay`

Recommended Mitigation

Add an early return in `FlashLeverage::_revertIfEffectiveLtvTooHigh` when both the collateral and loan are zero. A position with no collateral and no debt is trivially healthy and does not need an LTV check.

```

// FlashLeverage.sol, inside _revertIfEffectiveLtvTooHigh

uint256 amountCollateralInLoanToken = getCollateralValueInLoanToken(
    market,
    amountLeveragedCollateral
).scaleTo(loanTokenDecimals, Math.STANDARD_DECIMALS);

```

```

+ if (amountCollateralInLoanToken == 0 && amountLoan == 0) {
+   return;
+ }
+
uint256 effectiveLtv = amountLoan.divDown(amountCollateralInLoanToken);

```

SpiralStake: Fixed in [dadd33](#).

Cyfrin: Verified.

7.3.2 Oracle price fluctuations allow users to bypass or be overcharged yield fees in FlashLeverage::withdrawCollateral

Description: The yield fee calculation in `FlashLeverage::withdrawCollateral` compares `netPositionValue` (computed using the current oracle price) against `position.amountDepositedInLoanToken` (recorded using the oracle price at deposit time). Since these two values are derived from oracle prices at different points in time, price movements between deposit and withdrawal distort the yield calculation.

When a user deposits collateral, its loan-token-denominated value is recorded in `amountDepositedInLoanToken` using the oracle price at that moment. On withdrawal, the net position value is recalculated using the current oracle price. If the oracle price of the collateral has decreased relative to the loan token since deposit, `netPositionValue` will be lower, making it appear that no yield was generated — even if the underlying collateral actually accrued yield. Conversely, if the oracle price increased, the system interprets the price appreciation as yield and charges fees on it, even though it may not represent actual protocol yield.

Furthermore, certain yield-generating assets like vault tokens can have morpho markets that use DEX prices as the oracle. In such scenarios, the underlying vault can have a positive yield, while the dex price can show no profit due to the exchanges not exactly matching vault prices. In liquidity crunch scenarios, the dex prices can exceed the actual vault exchange rate, reporting a higher yield than actually realized.

```

uint256 netPositionValue = amountCollateralInLoanToken - amountLoan;
if (netPositionValue > position.amountDepositedInLoanToken) {
    uint256 yieldGenerated;
    unchecked {
        yieldGenerated = netPositionValue - position.amountDepositedInLoanToken;
    }
    // ... yield fee charged on yieldGenerated
}

```

Impact:

- **Fee bypass:** Users can time withdrawals during periods of lower oracle prices to make `netPositionValue` fall below `amountDepositedInLoanToken`, avoiding yield fees entirely on actual yield generated.
- **Overcharging:** If the oracle price increases (e.g., due to market volatility rather than yield accrual), users are charged yield fees on price appreciation that is not actual protocol yield.
- **Credit accumulation:** Depositing at a high price and partially withdrawing at a lower price inflates `amountDepositedInLoanToken` by more than the fair value, building up "credit" that shields future genuine yield from fees.

Proof of Concept:

1. User opens a correlated-pair position when oracle price is high. `amountDepositedInLoanToken` = 100.
2. The collateral generates actual yield worth 5 in loan token terms.
3. Oracle price drops such that `netPositionValue` = 98 at withdrawal time.
4. Since $98 < 100$, no yield is detected — user pays zero yield fee despite earning yield worth 5.
5. Alternatively: oracle price rises such that `netPositionValue` = 110. User is charged fee on $110 - 100 = 10$, but only 5 is real yield — user overpays.

Recommended Mitigation: Consider using a more robust oracle, such as vault-based oracles for reth/wsteth instead of the standard Morpho oracle, which might be chainlink-based. Otherwise, acknowledge this issue and document this behaviour.

Spiral Stake: Acknowledged. We accept that oracle price fluctuations can cause over- or under-charging of yield fees relative to actual yield accrual.

Cyfrin: Acknowledged.

7.3.3 Deposit fee on non-correlated markets can be bypassed by leveraging through the swap router

Description: In `FlashLeverage::leverage`, the deposit fee for non-correlated markets is charged only on `params.amountCollateral` — the collateral the user transfers in directly:

```
uint256 amountCollateral = _chargeDepositFeeIfNonCorrelated(
    market.collateralToken,
    params.marketId,
    params.amountCollateral
);
```

However, in `_handleLeverage`, the flash-loaned amount is swapped to collateral tokens via `_swapToken` and added to the position without any deposit fee:

```
uint256 amountSwappedCollateral = _swapToken(
    market.loanToken,
    market.collateralToken,
    amountLoan,
    swapData,
    minTokenOut
);
uint256 amountLeveragedCollateral = amountCollateral + amountSwappedCollateral;
```

A user can minimize `amountCollateral` (the directly deposited portion) and maximize `amountSwappedCollateral`, by routing the swap through a pool which they alone control. This way, they can control the price of the swap, feeding in the `amountCollateral` amount through `amountSwappedCollateral` instead. The collateral obtained via the swap is never subject to the deposit fee. Since `MAX_DEPOSIT_FEE` is 1% and `MAX_YIELD_FEE` is 10%, the deposit fee bypass is meaningful only when the yield fee does not apply (non-correlated markets don't have yield fees), making this a net loss of fee revenue for the protocol.

Impact: Users on non-correlated markets can avoid the deposit fee on the majority of their collateral by structuring the position to flow funds through the flash loan swap path rather than direct deposit. The protocol loses deposit fee revenue proportional to the amount routed through the swap.

Proof of Concept: Say collateral and debt are both priced at \$1.

User wants to supply \$20, and open a position of \$100 collateral and \$80 debt.

Normal operations: Normally, a user would send \$20 as `amountCollateral`, which would charge the deposit fee on \$20. Then \$80 debt would be flash-loaned, turned into \$80 collateral, supplied to morpho and \$80 debt would be used to pay off the flashloan. Fee was applied on \$20.

Manipulated operations: Here, the user creates a pool where

User now provides \$1 as `amountCollateral`. Then \$80 debt is flash-loaned and using this custom pool, converted into \$99 collateral token. This is then added as collateral, \$80 debt is borrowed to pay off the loan. Now, fee was only applied on the \$1 `amountCollateral`.

The user's pool eats a loss of \$19, but this was to be supplied anyways as `amountCollateral`. Effectively, the user supplied \$20 of extra collateral, but only \$1 of it was subjected to the fee.

Recommended Mitigation: Either put more checks on the router operations to ensure the swap goes through whitelisted pools, or charge the fee at the end based on the net position value (`collateral - debt`).

Spiral Stake: Acknowledged. The deposit fee is intentionally charged only on the user's direct collateral deposit. Exploiting this requires deploying a false pool to bypass the deposit fee, which isn't really worth the effort. We accept this as a known limitation.

Cyfrin: Acknowledged.

7.3.4 amountDepositedInLoanToken can be inflated via swap router manipulation to bypass yield fees

Description: In `FlashLeverage::_handleLeverage`, only the user-supplied `amountCollateral` portion is credited to `amountDepositedInLoanToken` — the flash-loaned portion that goes through the swap is not:

```
if (amountCollateral > 0) {
    uint256 amountDepositedInLoanToken = getCollateralValueInLoanToken(
        market,
        amountCollateral
    );
    position.amountDepositedInLoanToken += amountDepositedInLoanToken;
}
```

A user can exploit this by structuring the leverage call so that the swap router returns very few (or zero) collateral tokens, while the bulk of the collateral is passed in directly via `params.amountCollateral`. The user pre-acquires collateral tokens off-chain, passes them as `amountCollateral`, and sets swap parameters to route the flash loan through a custom pool that returns minimal output (the flash loan amount is effectively recycled back to repay itself through the custom pool's backdoor).

Since `amountDepositedInLoanToken` is inflated to cover nearly the full position value, the yield fee calculation in `withdrawCollateral` and `_handleDeleverage` sees little or no yield (`netPositionValue - amountDepositedInLoanToken` 0), and the user pays no yield fee despite genuine collateral appreciation.

Impact: Users on correlated markets can completely bypass yield fees by inflating their `amountDepositedInLoanToken` at position creation. The protocol loses all yield fee revenue from affected positions. Since the swap router parameters are fully user-controlled and only whitelisted router addresses are checked (not the underlying pool or calldata), any user can exploit this.

Proof of Concept:

1. User wants to open a leveraged position with 100 collateral tokens and 80 debt tokens on a correlated market.
2. User acquires 100 collateral tokens externally and calls `leverage` with `amountCollateral = 100` and `amountFlashLoan = 80`.
3. The swap routes the 80 loan token through a custom pool controlled by the user, which returns 0 collateral tokens (the 80 loan token is recoverable from the pool later, since it's controlled by the user, so no loss).
4. `amountLeveragedCollateral = 100 + 0 = 100`. The position is created with all collateral from the direct deposit.
5. 80 debt tokens are borrowed to pay off the flash loan.
6. `amountDepositedInLoanToken = getCollateralValueInLoanToken(market, 100)` — this records the full position value as "deposited."
7. When yield accrues and the user deleverages or withdraws, `netPositionValue` will approximately equal `amountDepositedInLoanToken`, so `yieldGenerated` 0 and no fee is charged.

Recommended Mitigation: Track `amountDepositedInLoanToken` based on the `netPositionValue` (collateral - debt) instead of relying on the user-specified input. Or put more checks on the router operations to ensure the swap goes through whitelisted pools

Spiral Stake: Acknowledged. Exploiting this requires a sophisticated setup, like deploying a false pool, which isn't really worth it. We accept this as a known limitation.

Cyfrin: Acknowledged.

7.3.5 Yield fee bypass during deleverage by routing collateral swap through an attacker-controlled pool

Description: This is similar to the issue `amountDepositedInLoanToken` can be inflated via swap router manipulation to bypass yield fees, but by using the `deleverage` function instead of the `leverage` function.

In `FlashLeverage::_handleDeleverage`, after repaying the Morpho loan and withdrawing all collateral, the collateral is swapped back to the loan token via `_swapToken`. The yield fee for correlated assets is computed based on `totalAmountReturned` — calculated as `amountSwappedLoanToken - amountLoan`. The user fully controls the `swapData` parameter, which determines the router and calldata for the swap.

Although the swap router must be whitelisted via `s_isSwapRouter`, the `extCalldata` passed to the router is entirely user-controlled. If a whitelisted router supports routing through arbitrary liquidity pools (as most aggregators like KyberSwap or Odos do), a malicious user can craft `swapData` to route the collateral swap through an attacker-controlled pool. This pool can accept all collateral tokens but return only enough loan tokens to cover the flash loan repayment (`amountLoan`), making `totalAmountReturned` zero or negligible. The attacker then recovers the collateral from their pool outside the transaction.

```
uint256 amountSwappedLoanToken = _swapToken(
    market.collateralToken,
    market.loanToken,
    amountLeveragedCollateral,
    swapData,           // <-- user-controlled, routes to attacker's pool
    minTokenOut         // <-- user sets this to amountLoan or 0
);

// totalAmountReturned = amountSwappedLoanToken - amountLoan  0
// yield fee = 0
```

Impact: Users can completely bypass yield fees on correlated-asset positions during deleverage. The protocol treasury receives no fee despite the position having generated yield. This represents a direct loss of protocol revenue for every correlated-asset position that deleverages using this technique.

Proof of Concept:

1. User opens a leveraged position on a correlated market (e.g., wstETH/WETH) with 10 ETH collateral
2. Over time, the position accrues yield — net position value exceeds `amountDepositedInLoanToken`
3. User deploys a custom liquidity pool or operates using an RFQ-based router like 1inch.
4. User calls `deleverage` with `swapData` that routes through their pool and sets `minTokenOut` to just enough to repay the flash loan
5. The pool accepts all collateral but returns only `amountLoan` worth of loan tokens
6. `totalAmountReturned = amountLoan - amountLoan = 0`, so no yield fee is charged
7. User recovers all collateral from their custom pool separately

Recommended Mitigation: Consider computing the yield fee based on the `netPositionValue` before the swap. However, this can run into an issue of overcharging the users, since some Morpho markets price their tokens based on asset manager or vault-reported values, but those rates aren't available on-chain, so they can never be reached via router swaps. Thus, users will be charged at rates that don't exist on-chain.

Another option would be to add more checks to the router/swapper, making sure they go through whitelisted pools only, to prevent such manipulation.

Spiral Stake: Acknowledged. This is a known edge case, but we consider it low severity.

Cyfrin: Acknowledged.

7.3.6 Malicious user can bypass yield fee by specifying non-zero `borrowShares` and inflated `amountRepay`

Description: The protocol exposes function `FlashLeverage::repay` allowing users to repay debt and reduce their position's loan-to-value ratio (LTV). The `repay` function internally calls the `MarketPositionManager::_morphoRe-`

pay function.

```
function repay(
    address user,
    uint256 positionId,
    uint256 amountRepay,
    uint256 borrowShares // Can be 0, mostly used for full loan repayment

    ... ..

    _morphoRepay(position.userProxy, market, amountRepay, borrowShares);
```

To satisfy the `exactlyOneZero` requirement in function `Morpho::repay`, function `_morphoRepay` checks if the `borrowShares` passed are equal to 0 and accordingly uses either the assets or the shares for repayment as observed in the snippet below.

```
function _morphoRepay(
    address userProxy,
    MarketParams memory marketParams,
    uint256 amount,
    uint256 borrowShares
) internal returns (uint256 assetsRepaid, uint256 sharesRepaid) {
    if (amount > 0) {
        _forceApprove(marketParams.loanToken, address(i_morpho), amount);

        address onBehalf = userProxy;
        (assetsRepaid, sharesRepaid) = i_morpho.repay(
            marketParams,
            borrowShares == 0 ? amount : 0, // <<
            borrowShares, // <<
            onBehalf,
            hex""
        );
    }
}
```

This allows a malicious user to specify `borrowShares` as a non-zero value, which would lead to the `assets` field to be ignored and evaluated to 0. When the execution context returns to the `FlashLeverage::repay` function, the position's `amountDepositedInLoanToken` is incremented by the inflated `amountRepay` instead of the actual amount that was repaid. Additionally, the `_morphoRepay` function leaves a hanging loan token approval to the Morpho contract.

```
_morphoRepay(position.userProxy, market, amountRepay, borrowShares);
position.amountDepositedInLoanToken += amountRepay;
```

Impact: Malicious user is able to bypass yield fees.

Proof of Concept: Let's take a simple scenario:

- Assume malicious Alice has a position with 20e18 collateral and 10e18 debt, where 1 collateral token = 1 loan token for simplicity with a Morpho LTV of 80%. At this point, `amountDepositedInLoanToken` = 20e18.
- Alice calls `repay` with a non-zero `borrowShares` value (assume her entire morpho position's debt shares) and 20e18 loan tokens as `amountRepay` even though her debt is 10e18 loan tokens.
- This would repay the entire debt but increment `amountDepositedInLoanToken` by 20e18 instead of 10e18. At this point, `amountDepositedInLoanToken` = 40e18.
- Assume the collateral token price appreciates leading to 1 collateral token = 2 loan token.
- In `withdrawCollateral`, since `netPositionValue > amountDepositedInLoanToken` (40e18 > 40e18) evaluates to false, the 20e18 loan token yield is never realized.

Recommended Mitigation: In `Morpho::repay`, when `borrowShares` is non-zero, consider using the actual amount of repaid assets returned from Morpho instead of the user input. Additionally, clear the hanging loan token approval after the repayment.

SpiralStake: Fixed in [2f292b](#).

Cyfrin: Verified.

7.3.7 Incorrect loan to collateral token decimal scaling leads to zero yield fees

Description: Function `FlashLeverage::withdrawCollateral` converts the yield fee calculated in loan token terms to collateral token terms. During this conversion, the functions `Math::mulDown` and `Math::divDown` are used from the `Math.sol` library.

```
// Convert fee from loanToken terms to collateralToken terms
// using the ratio: amountWithdraw / amountWithdrawInLoanToken
uint256 feeInCollateral = yieldFeeInLoanToken
    .mulDown(amountWithdraw)
    .divDown(amountWithdrawInLoanToken);
```

The issue is that if the loan and collateral tokens are both low-decimal tokens, the yield fee evaluates to 0. This occurs since `mulDown` divides the numerator by $1e18$ decimals.

```
uint256 internal constant ONE = 1e18; // 18 decimal places

function mulDown(uint256 a, uint256 b) internal pure returns (uint256) {
    uint256 product = a * b;
    unchecked {
        return product / ONE;
    }
}
```

Impact: Loss of yield fees.

Proof of Concept: Let's take the following example:

1. Assume collateral token = WBTC and loan token = USDC. WBTC has 8 decimals while USDC has 6 decimals on Ethereum Mainnet.
2. Let's say profit = 100 USDC. With a 10% yield fee, this evaluates to 10 USDC ($10e6$).
3. The `mulDown` operation will evaluate to: $10e6 * 1e8 / 1e18 = 1e15 / 1e18 = 0$.
4. Through this we observe how the `mulDown` scaling is not accurate when converting from loan token to collateral token terms.

Recommended Mitigation: Skip the mul/div down operations and directly do `(yieldFeeInLoanToken * amountWithdraw) / amountWithdrawInLoanToken`. The mul/div functions are unnecessary since it's not being done in 18 decimals.

SpiralStake: Fixed in [7f47e9](#).

Cyfrin: Verified.

7.3.8 Native tokens not supported in the router

Description: The `FlashLeverageRouter` contract swaps input tokens into the required collateral tokens, which will eventually be used to open the leveraged position. The issue is that this function takes in only ERC20 tokens and doesn't allow native ETH as input.

Impact: Since the `swapAndLeverage` is not payable, native tokens cannot be supplied as input. Thus, users cannot open leveraged positions with ETH and must first wrap them to WETH. The router contract should handle this part instead of the user.

Recommended Mitigation: Consider making the `swapAndLeverage` function payable and allowing ETH to be received and wrapped/swapped in order to open leverage positions.

Spiral Stake: Fixed in [92dfc0](#).

Cyfrin: Verified.

7.3.9 Owner can configure user proxies as swap routers to change Morpho positions and move user funds

Description: As per the [documentation](#), the owner of the `FlashLeverage` contract should not be able to move user funds or change Morpho positions.

What can the protocol owner do? Add or remove supported tokens and swap routers, adjust fees within the hardcoded caps, update the treasury address, and enable Manual Mode for user's position (upon requested by the user). The owner cannot move your funds, change your position, or bypass the liquidation buffer.

However, the owner can register any `UserProxy` as a swap router and execute operations on behalf of the user through `FlashLeverage`.

```
function setSwapRouter(address swapRouter, bool value) external onlyOwner {
    require(
        swapRouter != address(0),
        FLError.FlashLeverage__CannotBeZeroAddress()
    );

    _setSwapRouter(swapRouter, value);
}

function _setSwapRouter(address router, bool value) internal {
    s_isSwapRouter[router] = value;
}
```

Impact: The owner can escalate privileges to change user Morpho positions and move funds. While owners are restricted positions, this is still a case of privilege escalation, since owners are not meant to be able to manipulate user positions.

Proof of Concept: Let's take an example:

- Assume Morpho LTV is 50% and 1 collateral token is worth 1 loan token.
- Alice creates a position with 1000 collateral tokens and 50 loan tokens.
- This position is created on behalf of the user proxy through the `FlashLeverage` contract.
- Owner registers Alice's user proxy as a swap router.
- Owner performs a `FlashLeverage::leverage` call, which allows specifying the swap router and calldata to utilize. The owner specifies the swap router as Alice's user proxy. The calldata is specified as the function `UserProxy::execute` and parameter data as the function `Morpho::withdrawCollateral`.
- In `FlashLeverage::_handleLeverage`, internal function `MarketPositionManager::_swapToken` is used which executes the swap with the owner's provided calldata to withdraw 900 collateral tokens to the address specified by the owner in `withdrawCollateral`.
- Alice's position now only has 100 collateral tokens, leading to a significant loss of funds.

Recommended Mitigation: Disallow the owner from registering user proxies as swap routers.

SpiralStake: Fixed in [66b450](#).

Cyfrin: Verified.

7.4 Low Risk

7.4.1 Arithmetic underflow in `FlashLeverage::borrow` blocks borrowing capacity on correlated pairs after collateral appreciation

Description: On correlated markets, when collateral appreciates, the effective LTV drops and Morpho permits additional borrowing. However, `FlashLeverage::borrow` reverts with an arithmetic underflow at [FlashLeverage.sol:382](#):

```
position.amountDepositedInLoanToken -= amountBorrow;
```

`amountDepositedInLoanToken` tracks the user's capital contributions valued at **historical** oracle prices — it is set during `FlashLeverage::leverage` ([line 670](#)) and incremented during `FlashLeverage::supplyCollateral` ([line 355](#)) and `FlashLeverage::repay` ([line 418](#)). The `FlashLeverage::borrow` function subtracts the raw `amountBorrow` from this tracker, but `amountBorrow` is not bounded by `amountDepositedInLoanToken` — it is only bounded by Morpho's LTV check in `FlashLeverage::_revertIfEffectiveLtvTooHigh`.

When collateral appreciates, the effective LTV drops, allowing the user to borrow more than what was originally deposited in loan-token terms. Since `amountBorrow` can exceed `amountDepositedInLoanToken`, the subtraction underflows and the transaction reverts.

Concrete walkthrough (weETH/WETH correlated market, 60% appreciation):

Setup

- User deposits 100 weETH as collateral
- Opens a leveraged position at 70% target LTV
- `amountDepositedInLoanToken` is set to the initial collateral value at the original price

Time passes, weETH appreciates 60% relative to WETH

- Effective LTV drops significantly (from ~70% to ~44%)
- Morpho would allow substantial additional borrowing to bring LTV back up

User tries to borrow

The user calls `FlashLeverage::borrow` with an amount just above `amountDepositedInLoanToken`. The LTV check in `FlashLeverage::_revertIfEffectiveLtvTooHigh` passes because there is ample headroom, but:

```
position.amountDepositedInLoanToken -= amountBorrow
// amountDepositedInLoanToken - (amountDepositedInLoanToken + 1e18) → UNDERFLOW
```

The transaction reverts with a panic (error code 0x11), even though Morpho's own LTV constraints would have permitted the borrow.

Impact: Denial of service on borrowing. Users on correlated markets cannot access their full borrowing capacity after collateral appreciation. The protocol's LTV check passes, but the internal accounting underflows and blocks the transaction. This artificially restricts user access to capital that should be available according to the market's risk parameters. The severity increases with the degree of collateral appreciation. This will affect virtually every long-lived position when price for a pair appreciates above ~42%.

Proof of Concept: Place the following test in `test/poc/BorrowUnderflowDoS.t.sol`:

```
// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity 0.8.30;

import {Test, stdError} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC20Metadata} from "@openzeppelin/contracts/interfaces/IERC20Metadata.sol";
import {IMorpho, MarketParams, Id, Position} from "@morpho/interfaces/IMorpho.sol";
import {IOracle} from "@morpho/interfaces/IOracle.sol";
```

```

import {FlashLeverage} from "src/core/FlashLeverage/FlashLeverage.sol";
import {MarketConfig} from "src/core/structs/MarketConfig.sol";
import {LeverageParams} from "src/core/structs/LeverageParams.sol";
import {SwapData} from "src/core/structs/SwapData.sol";
import {LeveragePosition} from "src/core/structs/LeveragePosition.sol";
import {Math} from "src/core/libraries/Math.sol";

import {MockExtRouter} from "../mocks/MockExtRouter.sol";

contract BorrowUnderflowDoSTest is Test {
    using Math for uint256;

    address constant MORPHO = 0xBbBbBbBb9cC5e90e3b3Af64bdAF62C37EEFFCb;
    address constant TREASURY = 0xB90258b1F74a846F7941514C7c02Bb03EB249D5;
    bytes32 constant MARKET_ID = 0x37e7484d642d90f14451f1910ba4b7b8e4c3ccdd0ec28f8b2bdb35479e472ba7;

    FlashLeverage fl;
    IMorpho morpho;
    MockExtRouter mockRouter;
    MarketParams market;
    address user;
    uint8 loanDecimals;

    uint256 constant COLLATERAL = 100e18;
    uint256 constant TARGET_LTV = 70e16;

    function setUp() public {
        vm.createSelectFork(vm.envString("ETH_RPC_URL"), vm.envUint("BLOCK_NUMBER"));

        morpho = IMorpho(MORPHO);
        market = morpho.idToMarketParams(Id.wrap(MARKET_ID));
        loanDecimals = IERC20Metadata(market.loanToken).decimals();

        fl = new FlashLeverage(MORPHO, TREASURY);

        MarketConfig[] memory configs = new MarketConfig[](1);
        configs[0] = MarketConfig({marketId: MARKET_ID, isCorrelated: true});
        fl.addSupportedMarkets(configs);

        mockRouter = new MockExtRouter();
        fl.setSwapRouter(address(mockRouter), true);

        address supplier = makeAddr("Supplier");
        deal(market.loanToken, supplier, 100_000e18);
        vm.startPrank(supplier);
        IERC20(market.loanToken).approve(MORPHO, 100_000e18);
        morpho.supply(market, 100_000e18, 0, supplier, "");
        vm.stopPrank();

        user = makeAddr("user");
    }

    function test_borrowRevertsWithUnderflowOnCorrelatedPair() external {
        // 1. Open leveraged position
        _openLeveragedPosition();

        LeveragePosition memory pos = fl.getUserLeveragePosition(user, 0);
        uint256 deposited = pos.amountDepositedInLoanToken;
        console.log("amountDepositedInLoanToken after open:", deposited);

        // 2. Simulate 60% collateral appreciation via oracle mock
        uint256 originalPrice = IOracle(market.oracle).price();
        uint256 appreciatedPrice = (originalPrice * 160) / 100;
    }
}

```

```

vm.mockCall(
    market.oracle,
    abi.encodeWithSelector(IOracle.price.selector),
    abi.encode(appreciatedPrice)
);

// 3. Show LTV after appreciation and pick borrow amount
Position memory morphoPos = fl.getMorphoPosition(pos.userProxy, market);
uint256 collateralValueAfter = fl.getCollateralValueInLoanToken(market, morphoPos.collateral);
uint256 currentDebt = fl.getSharesValueInLoanToken(market, morphoPos.borrowShares);
uint256 currentLtv = (currentDebt * 1e18) / collateralValueAfter;

console.log("=== LTV after 60% appreciation ===");
console.log("Collateral value (loan token):", collateralValueAfter);
console.log("Current debt: ", currentDebt);
console.log("Current LTV (1e18 = 100%): ", currentLtv);

uint256 borrowAmount = deposited + 1e18;

uint256 projectedLtv = ((currentDebt + borrowAmount) * 1e18) / collateralValueAfter;
console.log("=== Projected borrow ===");
console.log("Borrow amount: ", borrowAmount);
console.log("Projected LTV after borrow: ", projectedLtv);

// 4. borrow() reverts due to underflow
vm.prank(user);
vm.expectRevert(stdError.arithmeticError);
fl.borrow(0, borrowAmount);
}

function _openLeveragedPosition() internal {
    deal(market.collateralToken, user, COLLATERAL);

    uint256 flashLoan = _calcFlashLoan(TARGET_LTV, COLLATERAL);
    SwapData memory swap = _buildSwap(market.loanToken, market.collateralToken, flashLoan);

    vm.startPrank(user);
    IERC20(market.collateralToken).approve(address(fl), COLLATERAL);
    fl.leverage(
        user,
        LeverageParams({
            marketId: MARKET_ID,
            amountCollateral: COLLATERAL,
            amountFlashLoan: flashLoan,
            swapData: swap,
            minTokenOut: 0
        })
    );
    vm.stopPrank();
}

function _calcFlashLoan(uint256 ltv, uint256 collateral) internal view returns (uint256) {
    uint256 colVal = fl.getCollateralValueInLoanToken(market, collateral)
        .scaleTo(loanDecimals, Math.STANDARD_DECIMALS);
    uint256 totalPos = colVal.divDown(Math.ONE - ltv);
    return (totalPos - colVal).scaleTo(Math.STANDARD_DECIMALS, loanDecimals);
}

function _buildSwap(address tokenIn, address tokenOut, uint256 amountIn) internal returns (SwapData
    ↪ memory) {
    uint256 oraclePrice = IOracle(market.oracle).price();
    uint256 amountOut = tokenIn == market.loanToken
        ? (amountIn * 1e36) / oraclePrice

```

```

        : (amountIn * oraclePrice) / 1e36;

    deal(tokenOut, address(mockRouter), amountOut);

    return SwapData({
        extRouter: address(mockRouter),
        extCalldata: abi.encodeCall(MockExtRouter.swap, (tokenIn, tokenOut, amountIn, amountOut))
    });
}
}

```

Run with: `forge test --match-test test_borrowRevertsWithUnderflowOnCorrelatedPair -vvv`

Recommended Mitigation: The same root cause as the `FlashLeverage::withdrawCollateral` underflow — `amountDepositedInLoanToken` is a historical-price tracker being decremented by a value that can exceed it.

Spiral Stake: Fixed in [1b485f](#).

Users are specifically prevented from borrowing on appreciated collateral value for correlated assets. They can always borrow against the appreciated collateral value for non-correlated assets.

Cyfrin: Verified.

7.4.2 Anyone can call `FlashLeverage::leverage` on behalf of any account, enabling dust position griefing and block reorg attacks

Description: The `FlashLeverage::leverage` function accepts an `onBehalfOf` parameter that allows any caller to open a leveraged position attributed to an arbitrary user. There is no authorization check to verify that `onBehalfOf` has consented to the position being created on their behalf.

```

function leverage(
    address onBehalfOf,
    LeverageParams calldata params
)
{
    // ...
    uint256 positionId = s_userLeveragePositions[onBehalfOf].length;
    s_userLeveragePositions[onBehalfOf].push(
        LeveragePosition({
            open: true,
            marketId: params.marketId,
            amountCollateral: amountCollateral,
            userProxy: address(0),
            amountDepositedInLoanToken: 0,
            amountReturnedInLoanToken: 0
        })
    );
    // ...
}

```

The only validation on `onBehalfOf` is that it is not `address(0)`. Any caller can supply any target address and create a position in that user's `s_userLeveragePositions` array.

Impact:

1. **Dust position griefing:** An attacker can create many small (dust) positions on behalf of a victim user, polluting their position array. This degrades the user experience by flooding `getUserLeveragePositions` with unwanted entries and forces the victim to interact with positions they never created.
2. **Block reorg position ID unpredictability:** Since `positionId` is derived from the length of the user's position array (`s_userLeveragePositions[onBehalfOf].length`), an attacker can front-run or exploit block reorganizations to insert positions before a legitimate user's transaction. This shifts the victim's expected

positionId, potentially causing subsequent calls to `deleverage`, `increaseLeverage`, `withdrawCollateral`, or other position-specific functions to target the wrong position.

Proof of Concept:

1. Alice intends to call `leverage` for herself. Before her transaction, her position array length is 3, so she expects `positionId = 3`.
2. Bob (attacker) observes Alice's pending transaction and front-runs it by calling `leverage(alice, ...)` with a dust collateral amount.
3. Bob's transaction executes first, creating a dust position at `positionId = 3` for Alice.
4. Alice's transaction executes and her position is created at `positionId = 4` instead of 3.
5. If Alice had a follow-up transaction (e.g., `increaseLeverage(3, ...)`) queued, it now targets Bob's dust position instead of her own.

Recommended Mitigation: Consider restricting `leverage` so that only the `onBehalfOf` address (or an explicitly approved operator) can open positions on their behalf. Otherwise, consider acknowledging and documenting this issue.

SpiralStake: Fixed in [f4211d](#).

Cyfrin: Verified.

7.4.3 `FlashLeverage::leverage` lacks reentrancy protection, allowing state manipulation during external swap calls

Description: The `FlashLeverage::leverage` function (as well as `deleverage`, `increaseLeverage`, and other external entry points like `supplyCollateral`, `borrow`, `repay`, and `withdrawCollateral`) do not have the `nonReentrant` modifier. Only the internal flash loan callback handlers `_handleLeverage` and `_handleDeleverage` are protected with `nonReentrant`.

During execution of `_handleLeverage` or `_handleDeleverage`, the `_swapToken` function performs a low-level `.call()` to an external swap router:

```
(bool success, ) = swapData.extRouter.call(swapData.extCalldata);
```

While the swap routers are whitelisted, the `calldata` is user-supplied and the router itself may delegate or transfer control to arbitrary addresses during swap execution (e.g., callback hooks in Uniswap V3/V4 swaps, or custom pool implementations). At this point, the `nonReentrant` lock is held for `_handleLeverage/_handleDeleverage`, but the external-facing functions that don't use flash loans — `supplyCollateral`, `borrow`, `repay`, and `withdrawCollateral` — can still be called because they never enter a `nonReentrant`-protected path.

This means an attacker gaining execution control during a swap can re-enter these unprotected functions and manipulate position state (e.g., `amountDepositedInLoanToken`, collateral balances) while the leverage or deleverage operation is still in progress and has not yet finalized its own state updates.

Impact: An attacker who can obtain execution control during a swap (via router callbacks or custom pool hooks) can re-enter unprotected functions like `supplyCollateral`, `borrow`, `repay`, or `withdrawCollateral` to manipulate position accounting while the `_handleLeverage` or `_handleDeleverage` operation is mid-execution. This violates the CEI pattern, allowing interactions with the contract in an incomplete state.

Recommended Mitigation: Add the `nonReentrant` modifier to all external state-modifying functions, not just the internal flash loan handlers.

Spiral Stake: Fixed in [d215ad](#).

Cyfrin: Verified.

7.4.4 `withdrawCollateral` reverts on division by zero when `amountWithdrawInLoanToken` rounds to zero

Description: In `FlashLeverage::withdrawCollateral`, the yield fee for correlated markets is converted from loan-token terms to collateral-token terms using:

```
uint256 feeInCollateral = yieldFeeInLoanToken
    .mulDown(amountWithdraw)
    .divDown(amountWithdrawInLoanToken);
```

`amountWithdrawInLoanToken` is computed by `getCollateralValueInLoanToken`, which multiplies the collateral amount by the oracle price via `mulDown` (dividing by `1e18`), then scales down from `18 + loanDecimals` to `loanDecimals`. When the collateral amount is very small (dust), and the loan token has low decimals (e.g., USDC with 6 decimals) and the collateral token has high decimals, this scaling can truncate the result to zero. `divDown` then attempts division by zero, which causes a panic revert.

This blocks the withdrawal entirely for dust collateral amounts on correlated markets that have accrued yield.

Impact: Users with correlated positions cannot withdraw dust amounts of collateral when yield has been generated. The transaction reverts unconditionally. While the impact is limited to very small withdrawal amounts, it can prevent users from fully closing out residual collateral in their positions.

Proof of Concept:

1. A correlated market uses a collateral token with 18 decimals and a loan token with 6 decimals (e.g., USDC).
2. A user has a position with accrued yield (`netPositionValue > amountDepositedInLoanToken`).
3. User calls `withdrawCollateral` with a dust `amountWithdraw` (e.g., 1 wei of collateral).
4. `getCollateralValueInLoanToken` computes the value: `1 * oraclePrice / 1e18`, then scales from 24 decimals to 6 decimals by dividing by `1e18`. The result truncates to 0.
5. `amountWithdrawInLoanToken = 0`, so `divDown(amountWithdrawInLoanToken)` reverts with a division-by-zero panic.

Recommended Mitigation: Add a check for `amountWithdrawInLoanToken == 0` before performing the fee calculation, skipping the fee when the withdrawal value rounds to zero in loan-token terms.

Spiral Stake: Fixed in [a803d7](#).

Cyfrin: Verified.

7.4.5 No functionality to remove or deprecate supported markets

Description: `FlashLeverage::addSupportedMarkets` allows the owner to add new markets, but there is no corresponding function to remove or deprecate them. Once a market is added to `s_markets`, it remains active indefinitely. The `isSupportedMarket` check only verifies that `collateralToken != address(0)`, which is always true for any added market.

Impact: If a supported market becomes unsafe — due to a compromised oracle, a depegged or exploited token, or deprecation on the underlying Morpho protocol — the owner has no way to prevent new positions from being opened on that market. Users could unknowingly open positions on markets that are no longer safe, potentially leading to loss of funds. The protocol has no emergency mechanism to disable leverage operations on a per-market basis.

Recommended Mitigation: Add a `s_marketEnabled` variable, which needs to be checked on all market operations. If disabled, only deleveraging completely should be allowed.

SpiralStake: Fixed in <https://github.com/spiral-stake/v2-core/commit/048febf6845d5dd013c3246053c2b913e77ea3f2>

Have added a `s_marketEnabled` mapping with an owner toggle. When a market is disabled, only new leverage calls will be blocked. Existing positions can still increaseLeverage, supplyCollateral, repay, withdrawCollateral, and deleverage normally so users can manage and unwind their positions without restriction.

Cyfrin: Verified.

7.4.6 amountDepositedInLoanToken becomes stale after manual mode operations or Morpho liquidations

Description: The amountDepositedInLoanToken field in LeveragePosition tracks how much value a user has deposited, and is used to compute yield for fee purposes. However, this value is only updated through FlashLeverage functions (leverage, supplyCollateral, repay, borrow, withdrawCollateral).

Two external pathways can modify the underlying Morpho position without updating amountDepositedInLoanToken:

1. **Manual mode:** When FlashLeverage::enableManualMode is called by the owner, the user gains direct access to UserProxy::execute, which forwards arbitrary calls to Morpho. The user can supply collateral, repay debt, borrow, or withdraw collateral directly on Morpho — none of which update amountDepositedInLoanToken.
2. **Morpho liquidations:** Any third party can liquidate an unhealthy position directly on Morpho against the UserProxy. This reduces the proxy's collateral and debt without FlashLeverage being aware, leaving amountDepositedInLoanToken unchanged.

Impact: The stale amountDepositedInLoanToken causes incorrect yield fee calculations during withdrawCollateral and deleverage. Depending on the direction of the discrepancy, the protocol may overcharge or undercharge yield fees. In extreme cases (e.g., after a large liquidation significantly reducing position value), amountDepositedInLoanToken could exceed the actual net position value, potentially making the position appear to have no yield when it does.

Recommended Mitigation: Document that amountDepositedInLoanToken is an approximation and may be inaccurate after manual mode operations or liquidations. Consider adding a function that allows recalibrating amountDepositedInLoanToken to the current net position value after such external events.

Spiral Stake: Acknowledged. Manual mode is a last resort intended only for rescuing user funds directly from Morpho in the event of an unforeseen bug, and it is not part of normal operation. We accept that yield fee calculations may be inaccurate afterward. This will be clearly documented.

Cyfrin: Acknowledged.

7.4.7 Unspent tokenIn not refunded after incomplete swaps in SwapManager::_swapToken

Description: In SwapManager::_swapToken, the full amountIn is approved to the external router before the swap call. Some routers (e.g., Uniswap V3's SwapRouter) may not consume the entire approved amount — for example, when the v3 pool runs out of liquidity, or if the sqrtPriceLimit specified is hit. After the swap, _swapToken only validates amountOut >= minTokenOut but never checks how much tokenIn was actually consumed or revokes the leftover approval. The unconsumed tokenIn remains sitting in the contract with an active approval to the external router.

```
_forceApprove(tokenIn, address(swapData.extRouter), amountIn);
(bool success, ) = swapData.extRouter.call(swapData.extCalldata);
require(success, "Swap Router Call Failed");

amountOut = _selfBalance(tokenOut) - balanceBefore;
require(amountOut >= minTokenOut, "minTokenOut Not Met");
```

The leftover tokenIn stays in the FlashLeverage contract and can be swept by a subsequent user's swap or consumed by the still-active router approval.

Impact: Users who trigger incomplete swaps lose the unconsumed input tokens. These tokens are left in the contract and can be claimed by the next caller or drained via the lingering approval. This results in a direct loss of funds for the affected user.

Recommended Mitigation: Consider refunding any tokenIn left in the contract after the swap.

This would also allow users to drain any funds present in this contract, so to prevent that, record the contract balance before taking in the tokens from the user, and return the difference (balanceAfter - balanceBefore). However, since this contract is not supposed to hold tokens, this behaviour (of users being able to claim the tokens from this contract for free) can just be documented and left as-is.

SpiralStake: Fixed in [2d9963](#).

Refunding any unconsumed `tokenIn` back to the user after the swap. Since the contract is not intended to hold any tokens, any remaining `tokenIn` balance is transferred out to the user.

Cyfrin: Verified.

7.5 Informational

7.5.1 FlashLeverage::borrow missing correlated-pair guard allows calls on correlated markets

Description: The FlashLeverage::borrow function's NatSpec explicitly states it "works only for non-correlated pairs", but the function body contains no check against FlashLeverage::s_isCorrelated:

```
/// @dev Only the position owner can call this function and works only for non-correlated pairs
function borrow(
    uint256 positionId,
    uint256 amountBorrow
) external validateAmount(amountBorrow) {
    address user = msg.sender;
    LeveragePosition storage position = s_userLeveragePositions[user][positionId];
    require(position.open, FLError.FlashLeverage__PositionAlreadyClosed);

    MarketParams memory market = s_markets[position.marketId];
    address userProxy = position.userProxy;

    _revertIfEffectiveLtvTooHigh(userProxy, market, 0, amountBorrow);
    _morphoBorrowViaProxy(userProxy, market, amountBorrow);

    position.amountDepositedInLoanToken -= amountBorrow;
    _transferOut(market.loanToken, user, amountBorrow);

    emit AdditionalBorrowed(user, positionId, amountBorrow);
}
```

Other functions in the contract (e.g. FlashLeverage::withdrawCollateral at (line 443), FlashLeverage::_chargeDepositFee at (line 775) properly branch on FlashLeverage::s_isCorrelated[position.marketId], but FlashLeverage::borrow does not.

This allows anyone with a correlated-pair position to call FlashLeverage::borrow.

Impact: Proof of Concept: Place the following test in test/ (e.g. test/Borrow.t.sol):

```
/// SPDX-License-Identifier: GPL-3.0-or-later
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {console} from "forge-std/console.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {IERC20Metadata} from "@openzeppelin/contracts/interfaces/IERC20Metadata.sol";
import {IMorpho, MarketParams, Id, Position} from "@morpho/interfaces/IMorpho.sol";
import {IOracle} from "@morpho/interfaces/IOracle.sol";

import {FlashLeverage} from "src/core/FlashLeverage/FlashLeverage.sol";
import {MarketConfig} from "src/core/structs/MarketConfig.sol";
import {LeverageParams} from "src/core/structs/LeverageParams.sol";
import {SwapData} from "src/core/structs/SwapData.sol";
import {LeveragePosition} from "src/core/structs/LeveragePosition.sol";
import {Math} from "src/core/libraries/Math.sol";

import {MockExtRouter} from "../mocks/MockExtRouter.sol";

contract CanBorrowCorrelatedPairsTest is Test {
    using Math for uint256;

    /// --- Mainnet constants ---
    address constant MORPHO = 0xBbBbBbBbBb9cC5e90e3b3Af64bdAF62C37EEFFCb;
    address constant TREASURY = 0xeB90258b1F74a846F7941514C7c02Bb03EB249D5;
    bytes32 constant MARKET_ID = 0x37e7484d642d90f14451f1910ba4b7b8e4c3ccdd0ec28f8b2bdb35479e472ba7;

    /// --- State ---
```

```

FlashLeverage fl;
IMorpho morpho;
MockExtRouter mockRouter;
MarketParams market;
address user;
uint8 loanDecimals;

uint256 constant COLLATERAL = 100e18;
uint256 constant TARGET_LTV = 70e16;

function setUp public {
    vm.createSelectFork(vm.envString("ETH_RPC_URL"), vm.envUint("BLOCK_NUMBER"));

    morpho = IMorpho(MORPHO);
    market = morpho.idToMarketParams(Id.wrap(MARKET_ID));
    loanDecimals = IERC20Metadata(market.loanToken).decimals;

    fl = new FlashLeverage(MORPHO, TREASURY);

    MarketConfig[] memory configs = new MarketConfig[](1);
    configs[0] = MarketConfig({marketId: MARKET_ID, isCorrelated: true});
    fl.addSupportedMarkets(configs);

    mockRouter = new MockExtRouter;
    fl.setSwapRouter(address(mockRouter), true);

    // Seed Morpho with loan-token liquidity
    address supplier = makeAddr("Supplier");
    deal(market.loanToken, supplier, 100_000e18);
    vm.startPrank(supplier);
    IERC20(market.loanToken).approve(MORPHO, 100_000e18);
    morpho.supply(market, 100_000e18, 0, supplier, "");
    vm.stopPrank;

    user = makeAddr("user");
}

/// @notice borrow has no correlated-pair guard, so it can be called on
///         correlated markets despite the NatSpec restriction.
function test_borrowSucceedsOnCorrelatedPair external {
    // --- 1. Open leveraged position on a correlated market ---
    _openLeveragedPosition;

    // Confirm the market is correlated
    assertTrue(fl.s_isCorrelated(MARKET_ID), "Market should be correlated");

    LeveragePosition memory posBefore = fl.getUserLeveragePosition(user, 0);
    uint256 depositedBefore = posBefore.amountDepositedInLoanToken;

    console.log("=== After opening position ===");
    console.log("amountDepositedInLoanToken:", depositedBefore);

    // --- 2. Simulate 50% collateral appreciation ---
    _appreciateCollateral(150);

    // --- 3. Borrow loan tokens on a correlated pair - should revert but doesn't ---
    uint256 borrowAmount = 50e18;

    vm.prank(user);
    fl.borrow(0, borrowAmount); // No revert - missing correlated-pair guard

    LeveragePosition memory posAfter = fl.getUserLeveragePosition(user, 0);
    uint256 depositedAfter = posAfter.amountDepositedInLoanToken;

```

```

        console.log("=== After borrowing on correlated pair ===");
        console.log("amountDepositedInLoanToken:", depositedAfter);
    }

    // ---- Helpers ----

    function _openLeveragedPosition internal {
        deal(market.collateralToken, user, COLLATERAL);

        uint256 flashLoan = _calcFlashLoan(TARGET_LTV, COLLATERAL);
        SwapData memory swap = _buildSwap(market.loanToken, market.collateralToken, flashLoan);

        vm.startPrank(user);
        IERC20(market.collateralToken).approve(address(f1), COLLATERAL);
        f1.leverage(
            user,
            LeverageParams({
                marketId: MARKET_ID,
                amountCollateral: COLLATERAL,
                amountFlashLoan: flashLoan,
                swapData: swap,
                minTokenOut: 0
            })
        );
        vm.stopPrank;
    }

    function _appreciateCollateral(uint256 pctOf100) internal {
        uint256 originalPrice = IOracle(market.oracle).price;
        uint256 appreciatedPrice = originalPrice * pctOf100 / 100;
        vm.mockCall(
            market.oracle,
            abi.encodeWithSelector(IOracle.price.selector),
            abi.encode(appreciatedPrice)
        );
    }

    function _calcFlashLoan(uint256 ltv, uint256 collateral) internal view returns (uint256) {
        uint256 colVal = f1.getCollateralValueInLoanToken(market, collateral)
            .scaleTo(loanDecimals, Math.STANDARD_DECIMALS);
        uint256 totalPos = colVal.divDown(Math.ONE - ltv);
        return (totalPos - colVal).scaleTo(Math.STANDARD_DECIMALS, loanDecimals);
    }

    function _buildSwap(address tokenIn, address tokenOut, uint256 amountIn) internal returns (SwapData
    ↪ memory) {
        uint256 oraclePrice = IOracle(market.oracle).price;
        uint256 amountOut = tokenIn == market.loanToken
            ? (amountIn * 1e36) / oraclePrice
            : (amountIn * oraclePrice) / 1e36;

        deal(tokenOut, address(mockRouter), amountOut);

        return SwapData({
            extRouter: address(mockRouter),
            extCalldata: abi.encodeCall(MockExtRouter.swap, (tokenIn, tokenOut, amountIn, amountOut))
        });
    }
}

```

Run with: `forge test --match-test test_borrowSucceedsOnCorrelatedPair`

Recommended Mitigation: Add a correlated-pair check at the top of borrow to enforce the documented invariant:

```
function borrow(
    uint256 positionId,
    uint256 amountBorrow
) external validateAmount(amountBorrow) {
    address user = msg.sender;
    LeveragePosition storage position = s_userLeveragePositions[user][positionId];
    require(position.open, FLError.FlashLeverage__PositionAlreadyClosed);
+   require(!s_isCorrelated[position.marketId], FLError.FlashLeverage__NotAllowedForCorrelatedPairs);

    MarketParams memory market = s_markets[position.marketId];
```

SpiralStake: Fixed in [63ba75](#).

Borrow is intentionally supported for correlated pairs as well. Updated the natspec to clarify this.

Cyfrin: Verified.

7.5.2 LeveragePosition::amountCollateral is not updated

Description: When a user supplies additional collateral via `FlashLeverage::supplyCollateral`, the `position.amountCollateral` field is never updated. This field is set once during leverage and remains stale after any subsequent collateral additions. While `amountDepositedInLoanToken` is correctly updated, the `amountCollateral` field in the `LeveragePosition` struct will not reflect the true amount of collateral the user has deposited.

Impact: Off-chain consumers (frontends, indexers) reading `getUserLeveragePositions` or `getUserLeveragePosition` will see an outdated `amountCollateral` value, potentially displaying incorrect position information to users. No on-chain logic depends on this field, so there is no direct financial impact.

Recommended Mitigation: Consider updating `position.amountCollateral` whenever the collateral amount is changed via `supply/withdraw collateral`, or acknowledge this behaviour and make sure off-chain components do not rely on this value.

Spiral Stake: Fixed in [fd646f](#).

Removed `amountCollateral` from the `LeveragePosition` struct entirely as it's un-utilised and remains stale. The initial collateral amount is emitted in the `LeveragePositionOpened` event for any off-chain tracking, if needed.

Cyfrin: Verified.

7.5.3 Collateral/repayments supplied directly to Morpho bypasses `amountDepositedInLoanToken` tracking and is treated as yield

Description: The `FlashLeverage` contract tracks how much value a user has deposited via `position.amountDepositedInLoanToken`. This value is used during `withdrawCollateral` and `deleverage` to determine whether yield was generated and to charge the appropriate yield fee.

However, if collateral is supplied directly to Morpho on behalf of a user's proxy (e.g., via `morpho.supplyCollateral` targeting the `UserProxy` address), the `amountDepositedInLoanToken` is never updated. The additional collateral increases the Morpho position's value but is not reflected in the `FlashLeverage` accounting. When the position is later closed or collateral is withdrawn, the difference between the net position value and `amountDepositedInLoanToken` is interpreted as yield, and yield fees are charged on it.

Impact: Users who supply collateral directly to Morpho (bypassing `FlashLeverage::supplyCollateral`) will have yield fees incorrectly charged on what was actually deposited capital, not generated yield. This results in a loss of funds for the user proportional to the externally supplied amount multiplied by `s_yieldFee`. The same also applies to the `repay` function, since users repaying debt directly through Morpho will also face the same issue, since `amountDepositedInLoanToken` is not updated.

Recommended Mitigation: Document this behavior clearly so users understand they must use `FlashLeverage::supplyCollateral` rather than interacting with Morpho directly.

Spiral Stake: Acknowledged. We are aware that supplying collateral or repaying debt directly to Morpho on behalf of a position's UserProxy bypasses the amountDepositedInLoanToken tracking, which would lead to incorrect yield fee calculations. Users should only interact via the FlashLeverage contract's supplyCollateral and repay functions. We will document this clearly and ensure the frontend enforces this path.

Cyfrin: Acknowledged.

7.5.4 Deposit fee charged on FlashLeverage::repay but not on FlashLeverage::deleverage, creating a fee inconsistency

Description: In FlashLeverage::repay, the loan token amount is passed through _chargeDepositFeeIfNonCorrelated before being used to repay debt on Morpho. This means that for non-correlated markets, a deposit fee (up to 1%) is deducted from the repayment amount.

However, in FlashLeverage::deleverage (via _handleDeleverage), the flash loan repays the full debt without any deposit fee being applied. This creates an inconsistency: a user who repays debt via repay and then withdraws collateral via withdrawCollateral pays a deposit fee on the repayment, while a user who simply calls deleverage to close the same position pays no such fee.

```
// In repay():
amountRepay = _chargeDepositFeeIfNonCorrelated(
    market.loanToken, position.marketId, amountRepay
);

// In _handleDeleverage(): no deposit fee on repayment
```

Impact: This creates an inconsistency in the fee model, since one path charges an extra fee while the other path doesn't, even though they lead to the same outcome.

Recommended Mitigation: Either apply the same fee model in both functions, or document this behaviour.

Spiral Stake: Acknowledged. This is intentional behavior and no changes are required and will be documented.

Cyfrin: Acknowledged.

7.5.5 Yield fee feeInCollateral can round down to zero, allowing fee bypass via small repeated withdrawals

Description: In FlashLeverage::withdrawCollateral, the yield fee is computed in loan-token terms (yieldFeeInLoanToken) and then converted to collateral-token terms:

```
uint256 feeInCollateral = yieldFeeInLoanToken
    .mulDown(amountWithdraw)
    .divDown(amountWithdrawInLoanToken);
```

mulDown divides by 1e18, which can truncate the intermediate result to zero when yieldFeeInLoanToken * amountWithdraw < 1e18. When feeInCollateral rounds to zero, the if (feeInCollateral > 0) check skips the fee transfer entirely. A user can exploit this by performing many small withdrawals, each sized so that the fee rounds to zero, effectively extracting yield without paying any fees.

Impact: The protocol's yield fee on correlated positions can be bypassed. A user can withdraw their full yield fee-free by splitting the withdrawal into sufficiently small amounts, resulting in lost fee revenue for the treasury.

Recommended Mitigation: Consider acknowledging and documenting this issue, since the fee is undercalculated by a single wei.

Spiral Stake: Acknowledged. Gas costs of many tiny withdrawals to exploit this far exceed any fee savings from rounding. Accepted as a known limitation.

Cyfrin: Acknowledged.

7.5.6 Pre-borrow LTV check underestimates actual post-borrow LTV due to Morpho's share-based rounding

Description: In `FlashLeverage::_handleLeverage`, `_revertIfEffectiveLtvTooHigh` is called *before* the collateral is supplied and the borrow is executed on Morpho:

```
_revertIfEffectiveLtvTooHigh(  
    userProxy,  
    market,  
    amountLeveragedCollateral,  
    amountLoan  
);  
// ...  
_supplyCollateralAndBorrowViaProxy(  
    userProxy,  
    market,  
    amountLeveragedCollateral,  
    amountLoan  
);
```

The check uses `amountLoan` (the raw asset amount) as the borrow liability. However, Morpho converts borrow assets to borrow shares using `toSharesUp` rounding, and then the liability is calculated by rounding up again, meaning the actual recorded borrow shares will represent a liability slightly higher than `amountLoan` (by up to 1 wei in asset terms). The effective LTV after the borrow on Morpho will therefore be marginally higher than what `_revertIfEffectiveLtvTooHigh` validated.

Impact: The actual post-leverage LTV can be a few wei higher than the validated LTV. Given the 2.5% LIQUIDATION_BUFFER, this rounding discrepancy (on the order of 1 wei) is practically negligible and does not pose a real risk of immediate liquidation. This is an informational accuracy concern only.

Recommended Mitigation: Consider acknowledging the issue given the 2.5% liquidation buffer. If desired for correctness, the LTV check could account for the share rounding by adding 1 wei to `amountLoan` before validation.

Spiral Stake: Acknowledged. The rounding discrepancy is at most a few wei in asset terms, which is negligible against the 2.5% liquidation buffer. No practical risk of liquidation from this.

Cyfrin: Acknowledged.

7.5.7 Flash loan repayment in `_handleDeleverage` can consume stuck/dust loan tokens held by `FlashLeverage`

Description: In `FlashLeverage::_handleDeleverage`, Morpho's flash loan repayment is covered by approving `amountLoan` for Morpho to pull via `safeTransferFrom`. The `totalAmountReturned` is calculated as `amountSwappedLoanToken - amountLoan`. However, Morpho pulls `amountLoan` from the contract's total loan token balance, not specifically from the swap proceeds. If the swap returns fewer loan tokens than `amountLoan`, any pre-existing loan token balance (dust from rounding, accidental transfers, or prior incomplete swaps) in the `FlashLeverage` contract is silently consumed to cover the shortfall.

Impact: Any loan tokens stuck or accidentally sent to the `FlashLeverage` contract can be consumed by a deleveraging user. The impact is limited since significant token accumulation in the contract is unlikely under normal operation, but it represents an unintended use of funds that don't belong to the deleveraging user.

Recommended Mitigation: Track the contract's loan token balance before and after the swap to determine the actual tokens available, and ensure only swap proceeds are used for flash loan repayment. Alternatively, add a sweep function for the owner to recover stuck tokens.

Spiral Stake: Acknowledged. Any dust or accidentally sent tokens sitting in the contract being consumed during deleverage is an acceptable tradeoff. We've also added a recover function for the owner to sweep any stuck tokens if needed.

Cyfrin: Acknowledged.

7.5.8 Incorrect comment in `_revertIfEffectiveLtvTooHigh` references `_handleDeleverage` instead of `_handleLeverage`

Description: The comment in `FlashLeverage::_revertIfEffectiveLtvTooHigh` states:

```
// proxy == address(0) when called from _handleDeleverage()
```

This is incorrect. The `userProxy` is `address(0)` only for new positions during `_handleLeverage`, where the proxy has not yet been created. By the time `_handleDeleverage` is called, the position always has an assigned proxy. The comment should reference `_handleLeverage` instead.

Impact: No functional impact. Misleading documentation may confuse developers during future maintenance.

Recommended Mitigation: Update the comment to:

```
// proxy == address(0) when called from _handleLeverage() for new positions
```

SpiralStake: Fixed in [c383cb](#).

Cyfrin: Verified.

7.5.9 `FlashLeverage::getMaxLtv` underflows for markets with `lltv` below `LIQUIDATION_BUFFER`

Description: `FlashLeverage::getMaxLtv` computes the maximum allowed LTV as:

```
return getLiqLtv(market) - LIQUIDATION_BUFFER;
```

`LIQUIDATION_BUFFER` is hardcoded to `25e15` (2.5%). If a Morpho market has an `lltv` less than or equal to 2.5%, this subtraction underflows, causing a revert. Since Solidity 0.8.30 has default overflow/underflow checks, any call to `_revertIfEffectiveLtvTooHigh` — and by extension `leverage`, `increaseLeverage`, `withdrawCollateral`, and `borrow` — will revert for such markets.

Impact: Markets with very low `lltv` values (2.5%) cannot be used with `FlashLeverage` even if added via `addSupportedMarkets`. All leverage operations will revert. While such low-LLTV markets are uncommon, Morpho supports arbitrary `lltv` values below `1e18`, so such markets can exist.

Recommended Mitigation: Make sure that a market's `lltv` exceeds `LIQUIDATION_BUFFER` in `addSupportedMarkets` to prevent onboarding incompatible markets.

Spiral Stake: Acknowledged. The governance-approved LLTVs on Morpho start at 38.5% for markets. No existing market has an LLTV anywhere close to 2.5%, and we won't be adding such markets in the future either.

Cyfrin: Acknowledged.

7.5.10 `UserProxy` implementation contract can be initialized by anyone

Description: The `UserProxy` implementation contract deployed in the `FlashLeverage` constructor lacks protection against direct initialization. Since `initialize` only checks `user == address(0)`, anyone can call `initialize` on the implementation contract itself (not the clones) and set themselves as the `user`.

```
function initialize(address _user) external {
    require(user == address(0), "UserProxy: Already Initialized");
    user = _user;
}
```

While this has no direct impact on cloned proxies (each clone initializes again), it is a best practice to prevent initialization of implementation contracts to avoid confusion or unexpected interactions.

Impact: Negligible. The implementation contract holds no funds and is not used directly for any position logic. However, an initialized implementation could be misleading during on-chain analysis.

Recommended Mitigation: Use the `openzeppelin.disableInitializers` in the implementation contract constructor, or initialize the implementation contract directly after deployment.

SpiralStake: Fixed in [c2fb74](#). The implementation contract is now initialized in the constructor itself, preventing anyone from calling initialize on it directly.

Cyfrin: Verified.

7.5.11 Unused error `FlashLeverage__CannotBorrowForCorrelatedPair` in `FLError` library

Description: The error `FLError::FlashLeverage__CannotBorrowForCorrelatedPair` is declared in `Error.sol` but is never used anywhere in the codebase. This suggests either dead code or a missing check in `FlashLeverage::borrow` (which per its `NatSpec` should restrict borrowing to non-correlated pairs only).

Impact: No runtime impact. Dead code reduces readability.

Recommended Mitigation: Either remove the unused error or implement the intended correlated-pair restriction in `FlashLeverage::borrow` using this error.

SpiralStake: Fixed in [f3a436](#).

Cyfrin: Verified.

7.5.12 `FlashLeverageRouter::swapAndLeverage` uses full balance instead of swap output, capturing pre-existing tokens

Description: In `FlashLeverageRouter::swapAndLeverage`, after the external swap call, `amountCollateral` is set to the contract's entire balance of the collateral token rather than the actual output of the swap:

```
leverageParams.amountCollateral = _selfBalance(market.collateralToken);
```

If the `FlashLeverageRouter` contract holds any pre-existing collateral tokens (e.g., from failed transactions, incomplete swaps, or airdropped tokens), those tokens are silently included in the caller's leverage position.

Impact: Any collateral tokens stuck in the router contract can be captured by the next user who calls `swapAndLeverage` on a market using that same collateral token. This is a theft-of-funds vector for any tokens that accumulate in the router.

Recommended Mitigation: Track the collateral token balance before and after the swap, and use only the difference:

```
uint256 collateralBefore = _selfBalance(market.collateralToken);

(bool success, ) = swapData.extRouter.call(swapData.extCalldata);
require(success, "Swap Router Call Failed");

leverageParams.amountCollateral = _selfBalance(market.collateralToken) - collateralBefore;
```

Spiral Stake: Acknowledged. The `FlashLeverageRouter` is not intended to hold any token balances. If someone benefits from dust or accidentally sent tokens sitting in the contract, that's an acceptable tradeoff.

Cyfrin: Acknowledged.

7.5.13 Implement consistent variable naming across contracts

Description: The `FlashLeverage`, `SwapManager` and `MarketPositionManager` contracts prepend the letter "s" and "i" before storage and immutable variables. However the `UserProxy` contract does not follow the same pattern.

`MarketPositionManager` contract:

```
IMorpho public immutable i_morpho;

mapping(bytes32 marketId => MarketParams) internal s_markets;

// Cached Loan Token Decimals to save gas by reducing external calls
mapping(address loanToken => uint8) internal s_loanTokenDecimals;
```


UserProxy contract:

```
/// @notice Address of the user who owns this proxy and can be initialized only once
address public user;
/// @notice Address of the FlashLeverage contract that controls this proxy
address public immutable flashLeverage;
/// @notice Address of the Morpho contract to borrow and repay
address public immutable morpho;
/// @notice Flag indicating whether manual mode is active for this proxy contract
bool public manualMode;
```

Recommended Mitigation: Implement consistency across all contracts by updating the UserProxy as such:

UserProxy contract:

```
/// @notice Address of the user who owns this proxy and can be initialized only once
address public s_user;
/// @notice Address of the FlashLeverage contract that controls this proxy
address public immutable i_flashLeverage;
/// @notice Address of the Morpho contract to borrow and repay
address public immutable i_morpho;
/// @notice Flag indicating whether manual mode is active for this proxy contract
bool public s_manualMode;
```

SpiralStake: Fixed in [34ef9a](#).

Cyfrin: Verified.

7.5.14 Missing event emission on critical state changes and operations

Description: The UserProxy contract does not emit events when critical state changes and operations occur in the contract.

```
function initialize(address _user) external {
    require(user == address(0), "UserProxy: Already Initialized");
    user = _user;
}

function enableManualMode() external {
    require(msg.sender == flashLeverage, "UserProxy: Unauthorised");
    manualMode = true;
}

function recover(address token) external {
    require(msg.sender == user, "UserProxy: Unauthorised");
    _transferOut(token, user, _selfBalance(token));
}
```

Multiple other instances exists in the SwapManager and FlashLeverage contracts.

SwapMananger:

```
function _setSwapRouter(address router, bool value) internal {
    s_isSwapRouter[router] = value;
}
```

FlashLeverage:

```
function addSupportedMarkets(
    MarketConfig[] memory marketConfigs
) external onlyOwner {
    uint256 marketConfigsLength = marketConfigs.length;
```

```

    for (uint256 i; i < marketConfigsLength; ++i) {
        MarketConfig memory marketConfig = marketConfigs[i];

        MarketParams memory market = i_morpho.idToMarketParams(
            Id.wrap(marketConfig.marketId)
        );

        _updateMarket(marketConfig.marketId, market);
        s_isCorrelated[marketConfig.marketId] = marketConfig.isCorrelated;
    }
}

function updateTreasury(address newTreasury) external onlyOwner {
    require(
        newTreasury != address(0),
        FLError.FlashLeverage__CannotBeZeroAddress()
    );

    s_treasury = newTreasury;
}

function updateYieldFee(uint256 newYieldFee) external onlyOwner {
    require(
        newYieldFee != 0 && newYieldFee <= MAX_YIELD_FEE,
        FLError.FlashLeverage__InvalidYieldFee()
    );

    s_yieldFee = newYieldFee;
}

function updateDepositFee(uint256 newDepositFee) external onlyOwner {
    require(
        newDepositFee <= MAX_DEPOSIT_FEE,
        FLError.FlashLeverage__InvalidDepositFee()
    );

    s_depositFee = newDepositFee;
}

```

Recommended Mitigation: Emit events across the codebase on critical state changes/operations

SpiralStake: Fixed in [398fd0](#).

Cyfrin: Verified.

7.5.15 Consider using named mappings to improve code readability

Description: Use named mapping for key-value fields in SwapManager to improve code readability.

```
mapping(address => bool) private s_isSwapRouter;
```

Recommended Mitigation: For example, use:

```
mapping(address router => bool approved) private s_isSwapRouter;
```

SpiralStake: Fixed in [8c8cc7](#).

Cyfrin: Verified.

7.5.16 FlashLeverage lacks pause functionality

Description: The FlashLeverage contract does not implement pause functionality to restrict user operations during emergency scenarios.

Recommended Mitigation: It is recommended to implement pause and unpause functionality on all user facing functionality in the FlashLeverage contract. Please note that this also comes with the risk of possible position liquidations if the contract stays paused for too long and users are not able to modify their positions in time.

Spiral Stake: Fixed in [d215ad](#).

Cyfrin: Verified.

7.6 Gas Optimization

7.6.1 Consider using custom errors instead of string error messages

Description: Multiple instances across the FlashLeverageRouter, SwapManager and UserProxy contracts use string error messages unlike the FlashLeverage contract utilizing custom errors. Custom errors are cheaper than require statements with strings since only 4 bytes needs to be stored in memory. In the case of string messages in require statements, Solidity has to store in memory and revert with at least 64 bytes.

```
function initialize(address _user) external {  
    require(user == address(0), "UserProxy: Already Initialized");  
    user = _user;  
}
```

Recommended Mitigation: Use custom errors in the respective contract to save gas and maintain consistency.

SpiralStake: Fixed in [be2733](#).

Cyfrin: Verified.